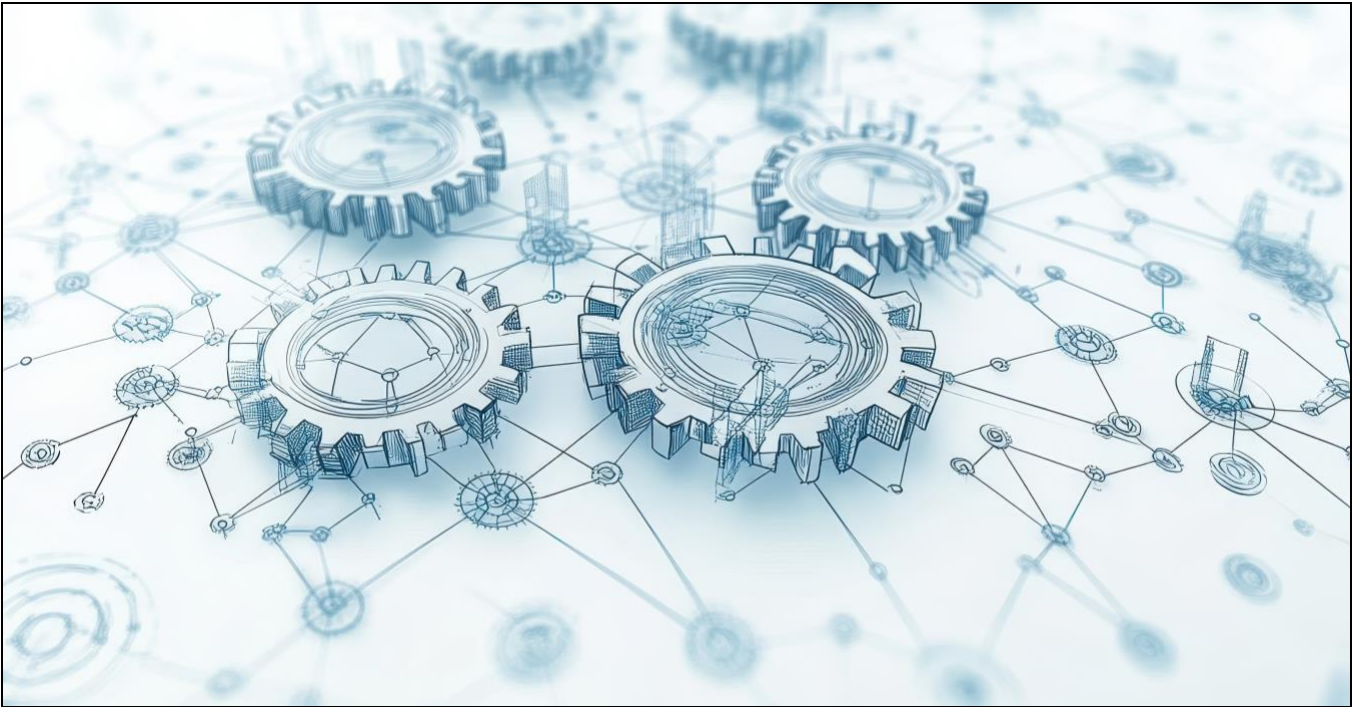


Level 4

Watsonx Orchestrate

Hands-on lab guide: watsonx Orchestrate API



Version 2.1

July 2026

LEGAL NOTICES

This information was developed for products and services offered in the USA. IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service. IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing IBM Corporation
North Castle Drive, MD-NC119 Armonk, NY 10504-1785
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you. This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice. Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk. IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you. Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All names and references for organizations and other business institutions used in this deliverable's scenarios are fictional. Any match with real organizations or institutions is coincidental. All names and associated information for people in this deliverable's scenarios are fictional. Any match with a real person is coincidental.

TRADEMARKS

IBM, the IBM logo, and [ibm.com](https://www.ibm.com) are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at <https://www.ibm.com/legal/copyright-trademark>.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates. INFOBLOX is a trademark of Infoblox Inc. or its affiliated companies, registered in the United States and other countries.

Microsoft®, Azure, Windows, Windows NT, Windows Server, Microsoft® Excel®, Microsoft® Word®, Microsoft® Outlook®, Microsoft® Excel® logo, Microsoft® Word® logo, Microsoft® Outlook® logo, Microsoft® Windows App, Microsoft® Windows App logo and the Windows logo are trademarks of Microsoft® Corporation in the United States, other countries, or both.

Salesforce and Salesforce logo are registered trademarks of SALESFORCE, INC. (CORPORATION; CALIFORNIA, USA).

Slack is a registered trademark of Salesforce, Inc (CORPORATION; DELAWARE, USA).

Red Hat®, JBoss®, OpenShift®, Fedora®, Hibernate®, Ansible®, CloudForms®, RHCA®, RHCE®, RHCSA®, Ceph®, and Gluster® are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Google, Google logo, Google Tasks, Google Cloud Platform, GMAIL, GMAIL logo are registered trademarks of GOOGLE LLC and this material is not endorsed by or affiliated with Google in any way.

Elasticsearch, Elastic search logo, Elastic, Elastic logo are registered trademarks of Elasticsearch BV (LIMITED LIABILITY COMPANY; NETHERLANDS) in United States, other countries or both.

Milvus, Milvus logo are registered trademarks of LF Projects, LLC (LIMITED LIABILITY COMPANY; DELAWARE, USA) in United States, other countries or both.

Meta, Meta logos, Llama, Llama logos are registered trademarks of META PLATFORMS, INC. (CORPORATION; California, USA) in United States, other countries or both.

Mistral, Mistral logo, Mistral AI, Mistral AI logo are registered trademarks of Mistral AI (SOCIÉTÉ PAR ACTIONS SIMPLIFIÉE (SAS); FRANCE) in United States, other countries or both.

PyCharm, PyCharm logo are registered trademarks of JetBrains s.r.o. (CORPORATION; CZECH REPUBLIC) in United States, other countries or both.

Bruno is a trademark held by Anoop M D. Bruno logo is sourced from OpenMoji. License: CC BY-SA 4.0

Apple, Apple logo, Apple store, App store, App store logo are registered trademarks of Apple Inc. (CORPORATION; CALIFORNIA, USA) in United States, other countries or both.

VMWARE, VMWARE logos are registered trademarks of VMware LLC (LIMITED LIABILITY COMPANY; Delaware, USA) in United States, other countries or both.

RANCHER, RANCHER logo, Rancher Desktop, Rancher Desktop logo are registered trademarks of SUSE LLC (LIMITED LIABILITY COMPANY; MASSACHUSETTS, USA) in United States, other countries or both.

Langfuse, Langfuse logo are registered trademarks of Langfuse GmbH (GESELLSCHAFT MIT BESCHRÄNKTER HAFTUNG (GMBH); Germany, Germany) in United States, other countries or both.

OpenMeteo is a registered trademark owned by Patrick Zippenfenig (Individual, Germany) in United States, other countries or both.

© Copyright International Business Machines Corporation 2025. This document may not be reproduced in whole or in part without the prior written permission of IBM.

Table of Contents

Introduction	1
Lab overview.....	1
What you will learn	1
How to get support	1
Prerequisites	1
Deploy the Weatherman agent	3
Authenticating using the API	4
Using an access token in the UI to authorize requests.....	6
Listing tenants.....	7
Updating the token to specify a tenant	9
Listing registered tools.....	12
Using Bruno for API testing.....	14
Listing tenants.....	17
Getting a tenant-specific access token.....	19
Listing registered agents.....	22
Chatting with an agent	24
Continuing a chat with an agent.....	28
Fetch a thread of messages	31
Summary	34
Appendix: Reset your environment	35

Introduction

Lab overview

This lab, building on knowledge and agents created in earlier labs, will show you how to authenticate with Orchestrate, list tenants, agents and tools, and fetch historical data, using the watsonx Orchestrate API.

What you will learn

Once you have completed this lab you will have developed skills enabling you to complete the following tasks:

- Authenticate with your Orchestrate environment.
- Use the API to get a list of tenants available in your environment.
- Use the API to discover tools available in your environment.
- Use the API to discover agents available in your environment.
- Chat with an agent using a single prompt.
- Continuing a conversation with an agent.
- Fetch a conversation thread.

How to get support

If you encounter an issue with this lab,

- IBM employees can open an IBM Support ticket at <http://ibm.biz/support>.
- Business Partners can request help at the [Partner Plus Support](#) website.

Prerequisites

Make sure the following prerequisites are satisfied before proceeding with this lab:

- You have performed the Preparation Lab guide including the environment validation step

- You have your local ADK 2.0 environment up and running (Orchestrate server started after validating the environment in the preparation guide OR after having performed another ADK related lab)

Note: If for some reason your server is not started correctly, reset your server following [the Reset your environment](#) appendix.

The API used in this lab are using web-services based on the OpenAPI Specification (OAS), basic knowledge of the format is not required but it is useful to improve your own knowledge and to extend agents behavior using tools that leverage web-services.

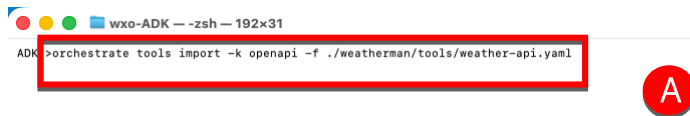
Extensive OAS resources are available online, this link from the OpenAPI initiative provides further details (<https://www.openapis.org/what-is-openapi>).

Deploy the Weatherman agent

In this section you will import a tool and deploy the Weatherman agent that was used in an earlier lab. Open a terminal window and execute the following commands.

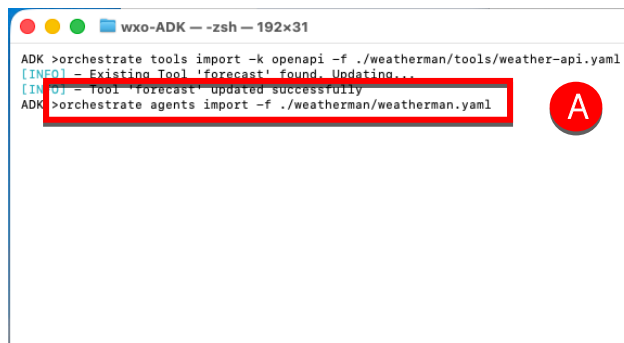
1. Open your ADK terminal as described in the preparation guide. Refer to [the Reset your environment appendix](#) if your server is not started.
2. Enter the following command (A) into the terminal to create a tool using the API specification:

```
orchestrate tools import -k openapi -f ./weatherman/tools/weather-api.yaml
```

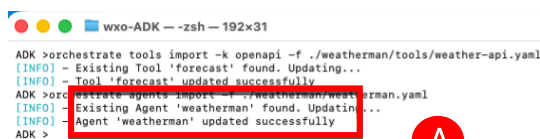


3. Enter the following command (A) into the terminal to create the agent:

```
orchestrate agents import -f ./weatherman/weatherman.yaml
```



4. Check that your agent is correctly imported (A).

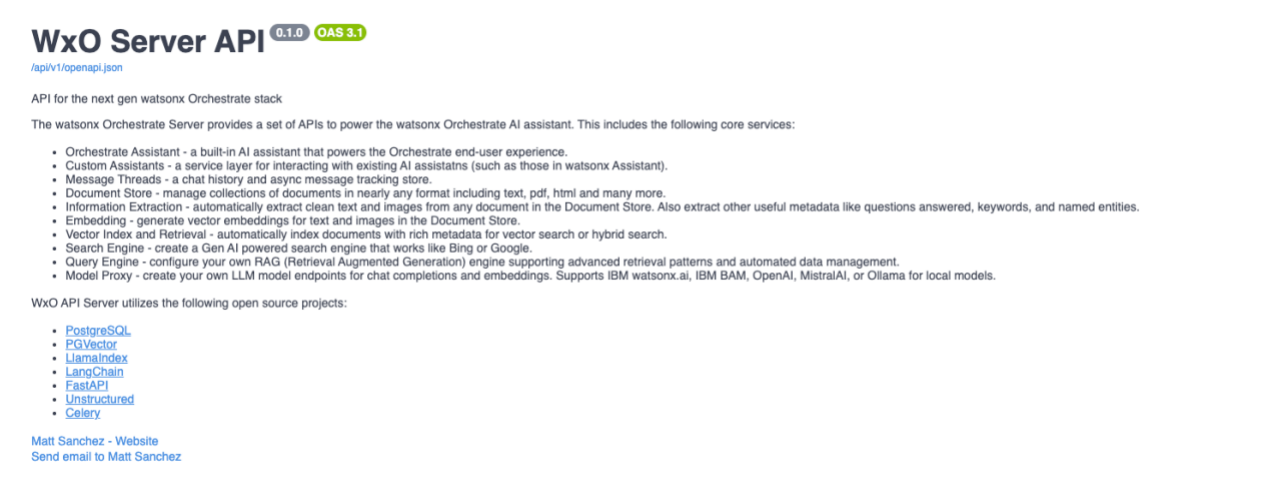


In a later section you will interact with the agent using the API.

Authenticating using the API

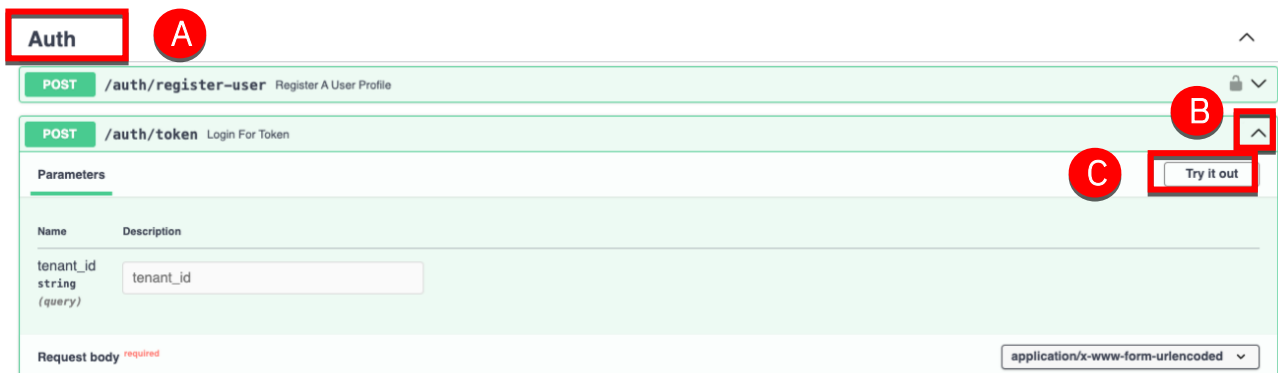
In this section you will be using the API to authenticate a user with watsonx Orchestrate ADK API. A token is required to communicate with the API endpoints and after authenticating successfully, you will receive a token that can be used to authenticate the requests to other API endpoints.

1. Open the browser, type <http://localhost:4321/docs> into the address bar and press enter. On the website that has opened, you will see the API documentation indicating that the ADK API is running successfully. This is an auto-generated API documentation and test page which was generated according to the OpenAPI specification. You should now see a web page similar to the picture below:



Note: Please make sure you are using the browser from inside your VMware image, not from your personal computer.

2. Within the web page, locate the **Auth** section (A) and the **Login For Token** endpoint, and click the expand button (B) and then click on **Try it out** (C).



It is necessary to authenticate with the server before commands can be issued using the API.

- Request body

required

application/x-www-form-urlencoded

grant_type

string | (string | null)

pattern: "password\$"

☐ Send empty value

username

string

password

string(\$password)

scope

string

☒ Send empty value

client_id

string | (string | null)

☐ Send empty value

client_secret

string | (string | null)(\$password)

☐ Send empty value
- A

B

C
- Execute

- Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
'http://localhost:4321/api/v1/auth/token' \  
-H 'accept: application/json' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
-d 'grant_type=password&username=xxo.archer@40ibm.com&password=watsonx&scope=client_idstring&client_secret=string'
```

Request URL

http://localhost:4321/api/v1/auth/token

Server response

Code

200

A

B

Response body

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJmMjQyZWZkZi0wZG5LTRlYWUyYjJkNy02NWl0eWl0YyRlMTY1LCl1c2VybmFtZSI6Imd4byShcmNoZXJAJ0JlNmVhSIsImF1ZCI6ImFidGh1bnRpYzF0ZmZkZiFQ.C1pKhYVdadGz4UB-jlQBZ-gECvkn1N9x4yEPFY73CVU".  
  "token_type": "bearer" }  
}
```

Download

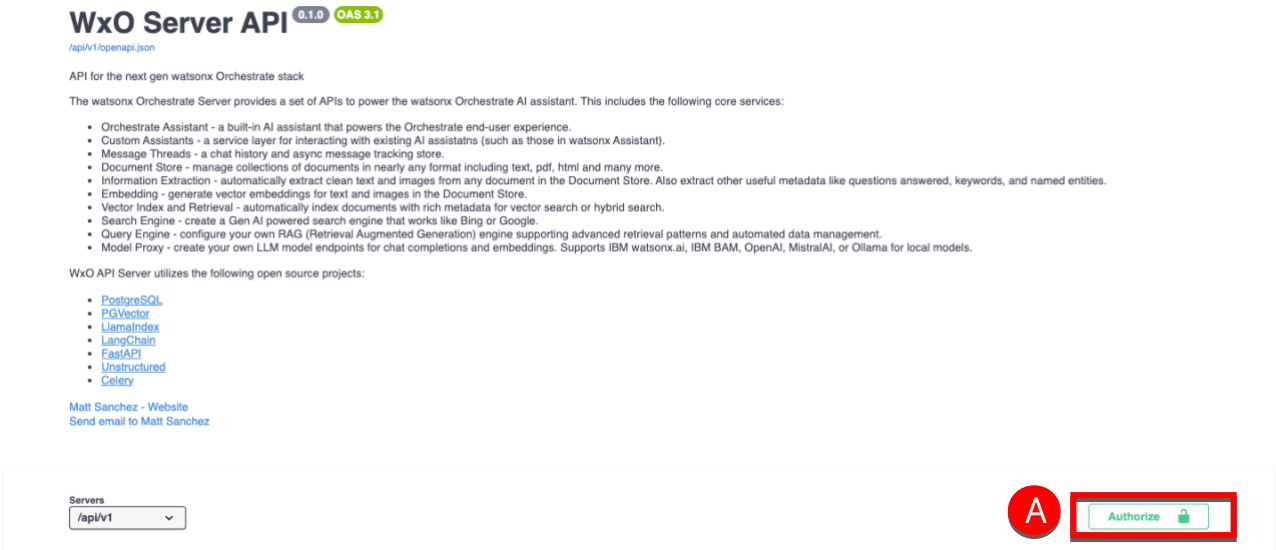
Response headers

```
access-control-allow-credentials: true  
access-control-allow-origin: *  
content-length: 256  
content-type: application/json  
date: Thu, 26 Jun 2025 07:11:08 GMT  
server: uvicorn
```

Using an access token in the UI to authorize requests

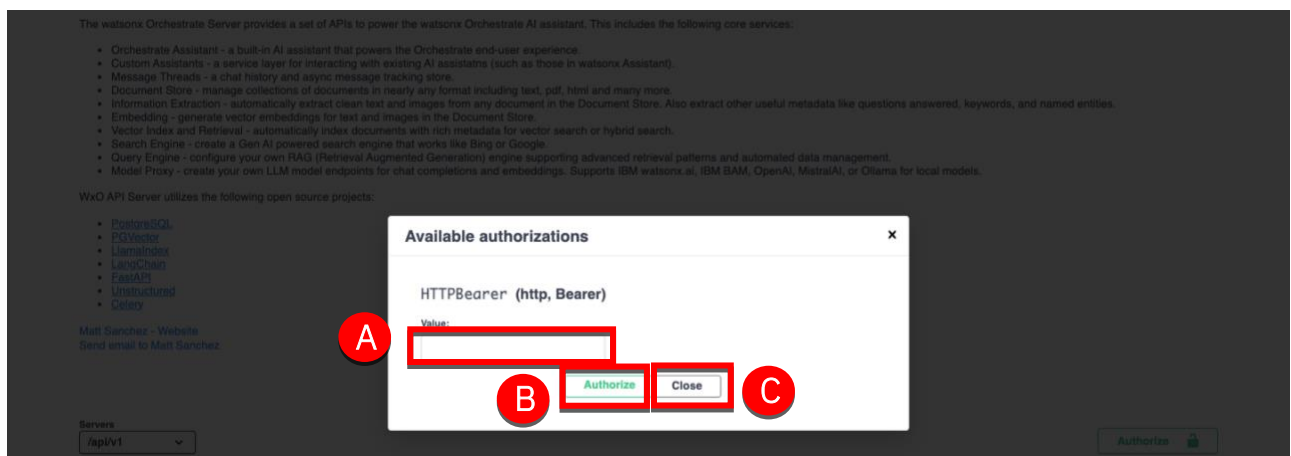
All API endpoints require authorization. In this step we will use the token obtained in the former steps, to authorize requests to a specific tenant and communicate with other API endpoints.

1. Navigate to the top of the page and click the **Authorize** button (A).



2. In the dialog that opens, paste your access token into the **value** field (A) from the former step and click **Authorize** (B) and finally click **Close** (C).

Note: If you have already logged in once, you will need to first click the **Logout** button before being able to update the **Value** field.

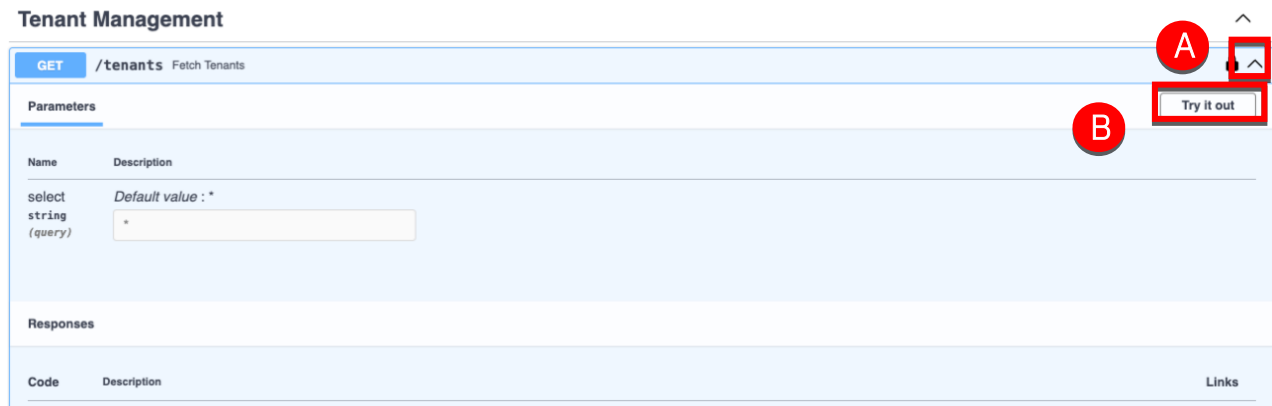


Congratulations! From now on, each API request from the UI will include an authorization header with the access token specified as a bearer token.

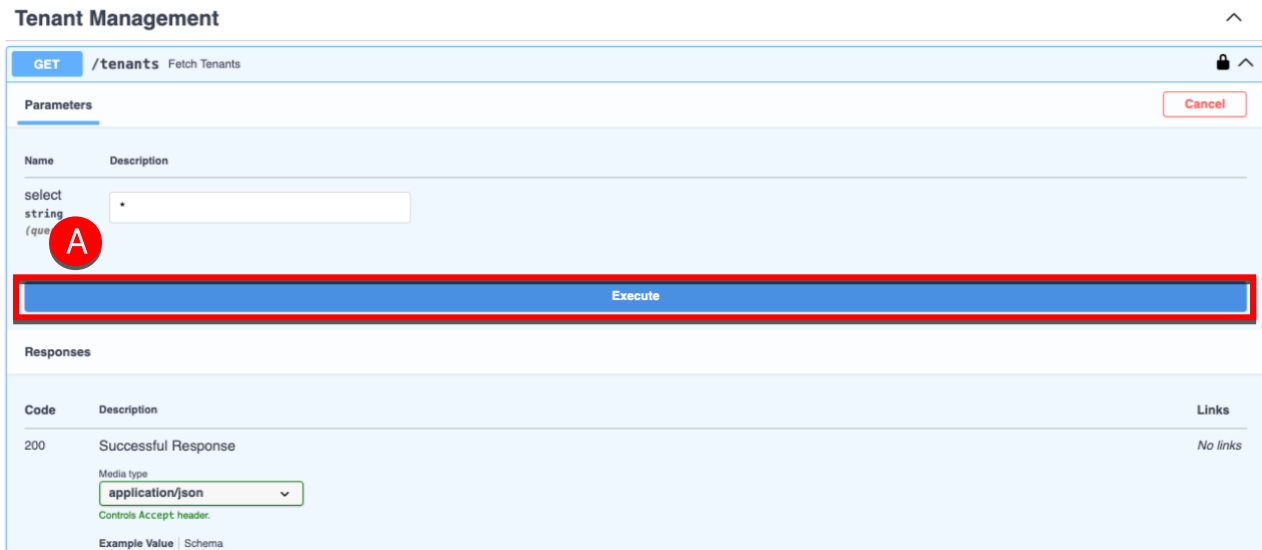
Listing tenants

In this section, we will be querying the API to get the list of tenants available in your environment. Each tenant is an isolated space for creating & managing agents, tools, chat and usage tracking. By having different tenants, teams can collaborate on agent-based solutions in a contained manner and separate projects or workspaces which enables tighter control on which agents and tools are available within each tenant.

1. Locate the **Tenant Management** section and within it the item **Fetch Tenants**, click on the item's expander button (A), then click **Try it out** (B).



2. Next, we will send an HTTP GET request to the API. Locate the **Execute** button, click it (A) and wait for the response.



- [illegible]

Updating the token to specify a tenant

For some operations the API requires authentication that includes a specific tenant identification, in this section will re-authorize using the tenant id obtained in the [former section](#).

1. Locate the **Auth** section and within it the **Login For Token** endpoint and click the expand button (A) and then click on **Try it out** (B).

Since you're returning to this section, you may find that it's already expanded and only shows the Cancel and Reset buttons, as illustrated below. If that's the case, you can move on to the next step.

The screenshot shows the API Explorer interface for the 'Auth' section. The 'Login For Token' endpoint (POST /auth/token) is selected. The 'Parameters' tab is active, displaying a table with columns 'Name' and 'Description'. A query parameter 'tenant_id' is listed with a text input field containing 'tenant_id'. There are 'Cancel' and 'Reset' buttons at the top right of the parameters section.

This screenshot shows the same API Explorer interface, but with red annotations. Circle A points to the expand button (a small upward arrow) at the top right of the endpoint bar. Circle B points to the 'Try it out' button, which is located at the bottom right of the parameters section. The 'Request body' section is also visible, showing a JSON schema for the request body.

2. Type the value of your Item 6 from the prep guide credentials table into the **username** field (A) and the value of Item 7 into the **password** field (B), paste the tenant id we saved in the former Listing Tenants section into the **tenant_id** field (C) and click Execute (D).

Please note it is possible that the username and password fields might be pre-populated based on the former request. The tenant_id may match the example below, or be “unknown”.

Auth

POST /auth/register-user Register A User Profile

POST /auth/token Login For Token

Parameters

Name	Description
tenant_id string (query)	a00ef710-70a9-4c73-9b74-aa8cff3af763

Cancel

Reset

Request body required

application/x-www-form-urlencoded

grant_type
string | (string | null)
pattern: "passwords"

password

☐ Send empty value

username * required
string

wxo.ard

A

password * required
string

wals

B

scope
string

scope

☒ Send empty value

client_id
string | (string | null)

string

☐ Send empty value

client_secret
string | (string | null)

string

☐ Send empty value

D

Execute

Responses

- Execute**

Responses

```
curl -X 'POST' \
'http://localhost:4321/api/v1/auth/token' \
-H 'accept: application/json' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-d 'grant_type=password&username=exo.archer%40ibm.com&password=watsonx&scope=&client_id=string&secret=string'
```

Request URL
`http://localhost:4321/api/v1/auth/token`

Server response

Code	Details
200	Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJMjQyZWFrZl0wZGM5LTRlYUUiLCJpc2VybWtZSI6Ind4byShcmNoZXJJadNkLnNbSisImF1ZC16ImlmaDghbmRpYTZfO2MqIFQ.CigKhYYdodgz4UB-j1QBZ-gECvknINgwyEPFY73CVU".
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 256
content-type: application/json
date: Thu, 26 Jun 2025 07:12:08 GMT
server: uvicorn
```

Responses

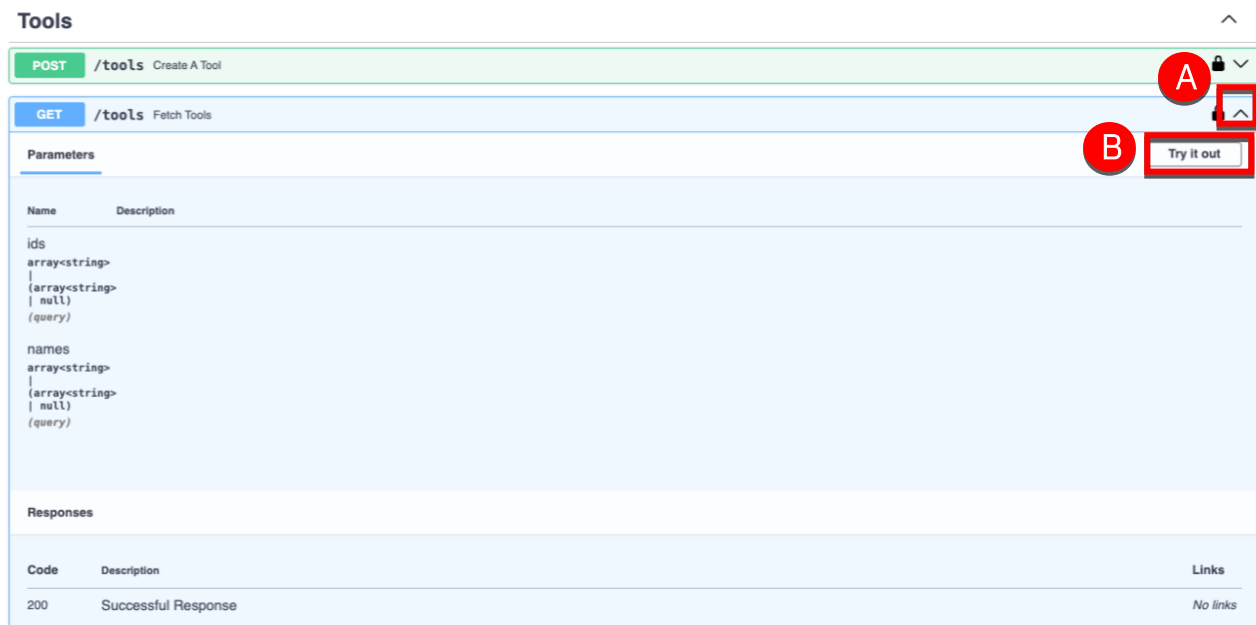
Code	Description	Links

You are now logged in with a tenant access token and can run commands against a specific tenant. This ADK can support connections to multiple different tenants, a local Developer Edition tenant and multiple remote tenants.

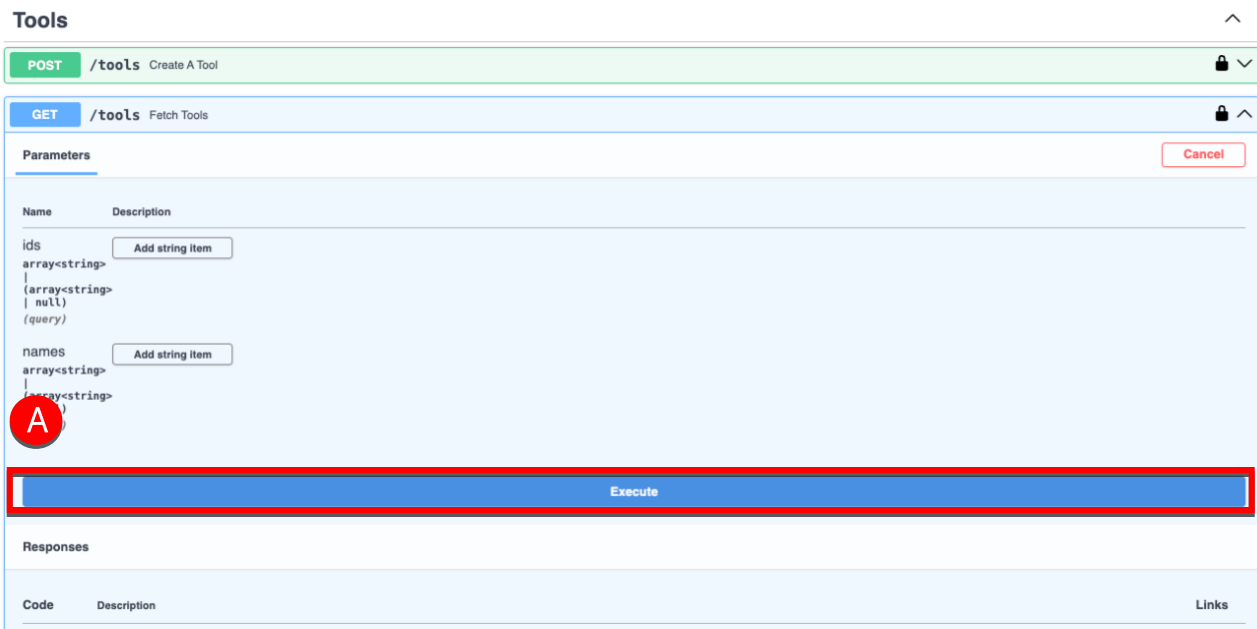
Listing registered tools

In this section, we will be querying the API to get the list of tools available in your environment. Tools registered in your environment can be referenced and used by agents which are in the same tenant.

1. Locate the **Tools** section and within it the item **Fetch tools**, click on the item's expander button (A), then click **Try it out** (B).



2. Next, we will send an HTTP GET request to the API. Locate the **Execute** button, click it (A) and wait for the response.



- Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:4321/api/v1/tools' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJmYjQyZWZkZi0wZGM5LTRlYWUtYjkyNzI0NDI0MTA7RmITMTYtYjIyNmFtZSI6ImF1dGh1bnVycy5jbG9jaW50eSIsImVudCkiOiJkaW4uYm9keSIsIm54IjoiMjAyNS05LTAxOjE1OjUzOTUuMjUifQ=='
```

Request URL

```
http://localhost:4321/api/v1/tools
```

Server response

CodeDetails

200

Response body

```
[{"id": "01fe651c-d9a0-4b38-a6eb-f82867289f0c", "tenant_id": "a00ef710-70a9-4c73-9b74-a08cff3af763", "description": "Weather information for given WGS84 latitude and longitude coordinates.", "created_on": "2025-06-26T07:43:42.590035Z", "created_by": "f242eadf-0dc9-4eae-b2d7-65b09b4b4b16", "updated_at": "2025-06-26T07:43:42.590035Z", "tags": null, "name": "forecast", "permission": "read_only", "input_schema": {"type": "object", "properties": {"query_latitude": {"type": "number", "title": "latitude", "description": "WGS84 coordinate", "format": "double", "in": "query", "aliasName": "Latitude"}, "query_longitude": {"type": "number"
```

Response headers

```
content-length: 2391
content-type: application/json
date: Tue, 01 Jul 2025 07:21:56 GMT
server: uvicorn
```

Responses

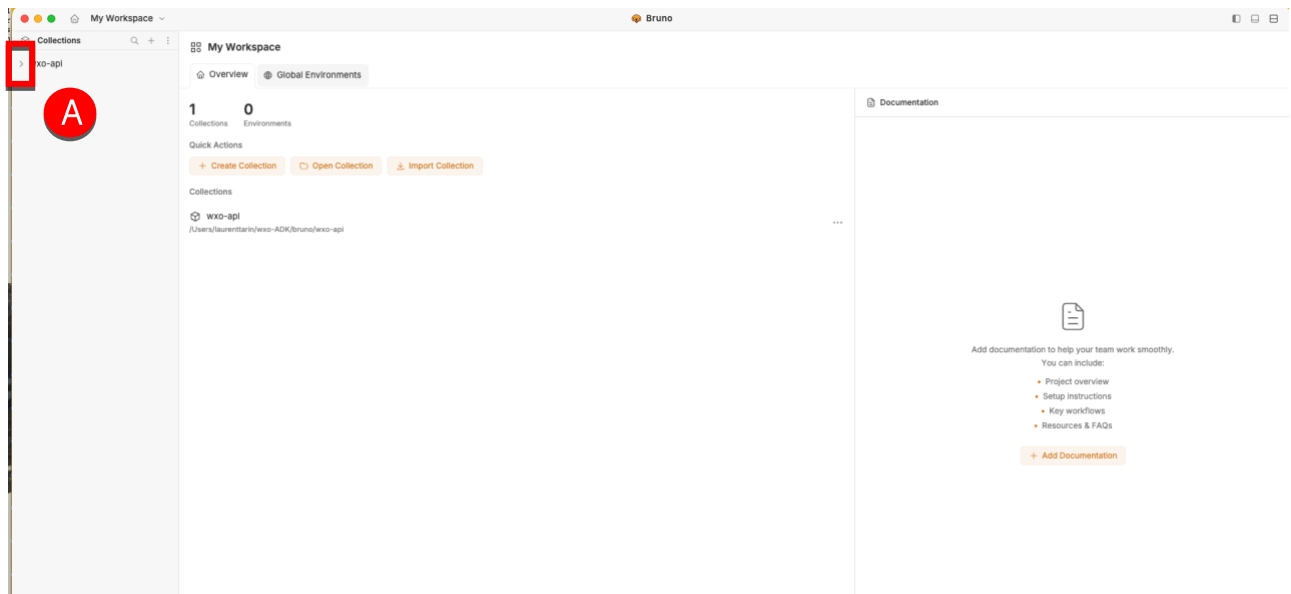
CodeDescriptionLinks

200Successful ResponseNo links

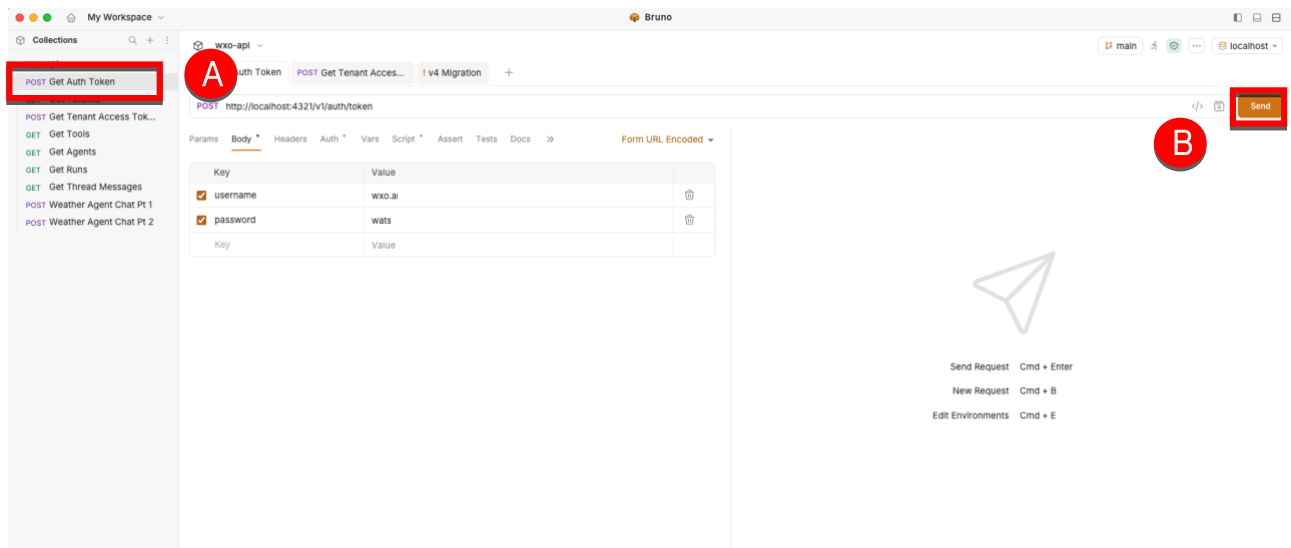
Using Bruno for API testing

In this section, we will be exploring the API using an API client called Bruno which is a fast and lightweight tool for testing APIs. In the following section we will login using tenant-specific authentication token which Bruno will save and use for further interactions with the API.

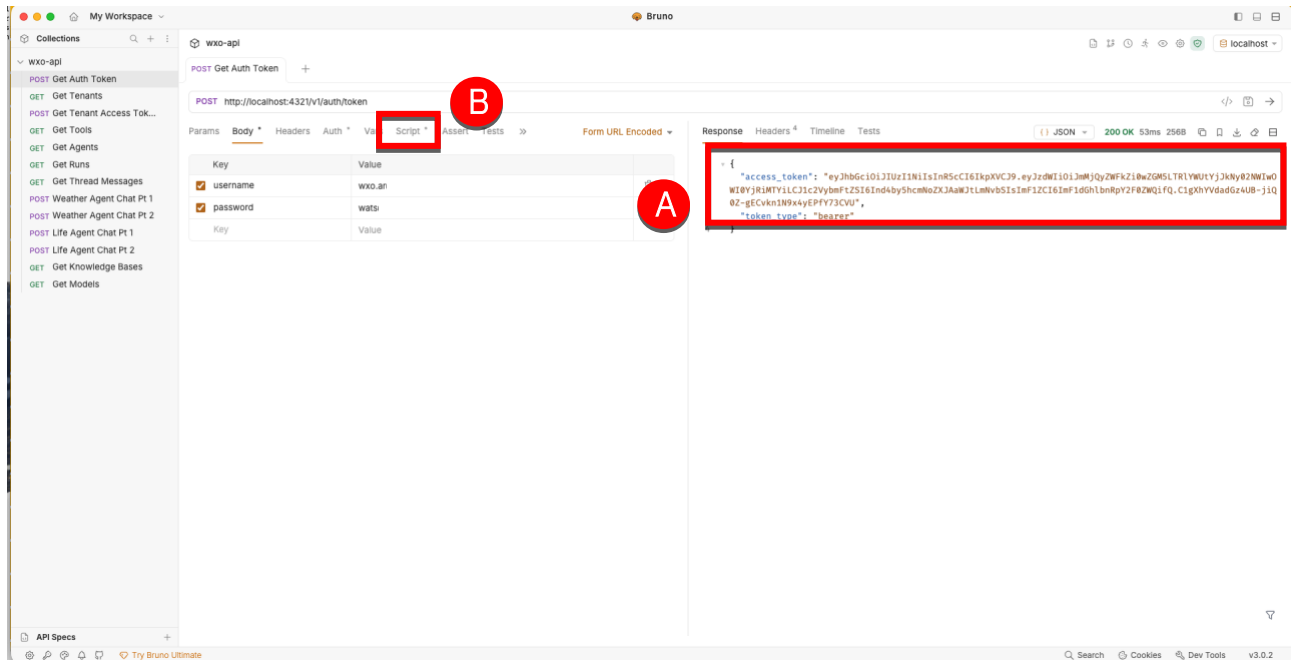
1. Start the Bruno application from your local computer.
2. Once Bruno is open, on the left-side navigation bar locate the **wxo-api** item and click the item's expander button to open it (A).



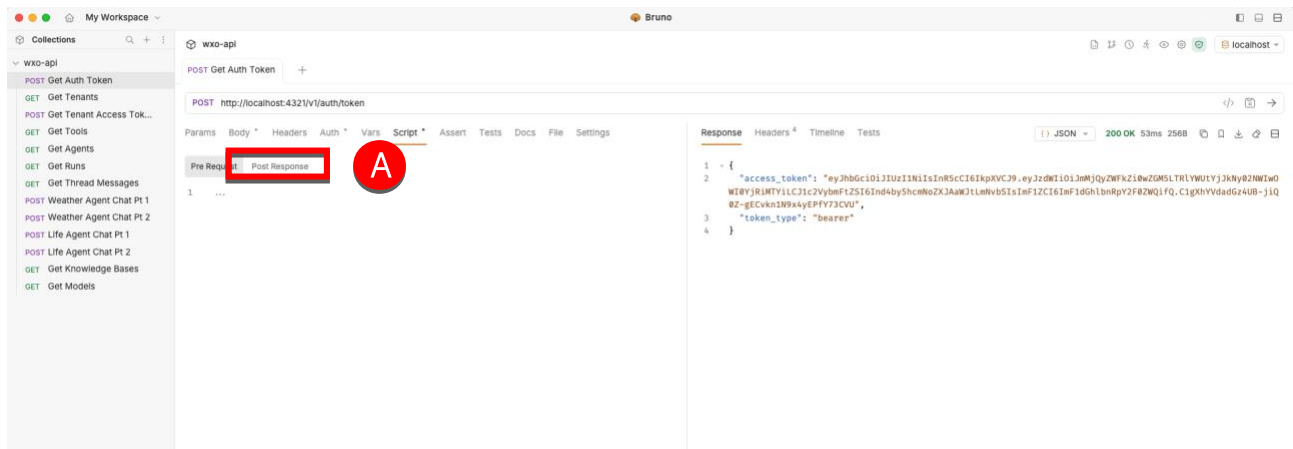
3. In the expanded list, click the **Get Auth Token** (A) item to open it, then click the Send button (B).



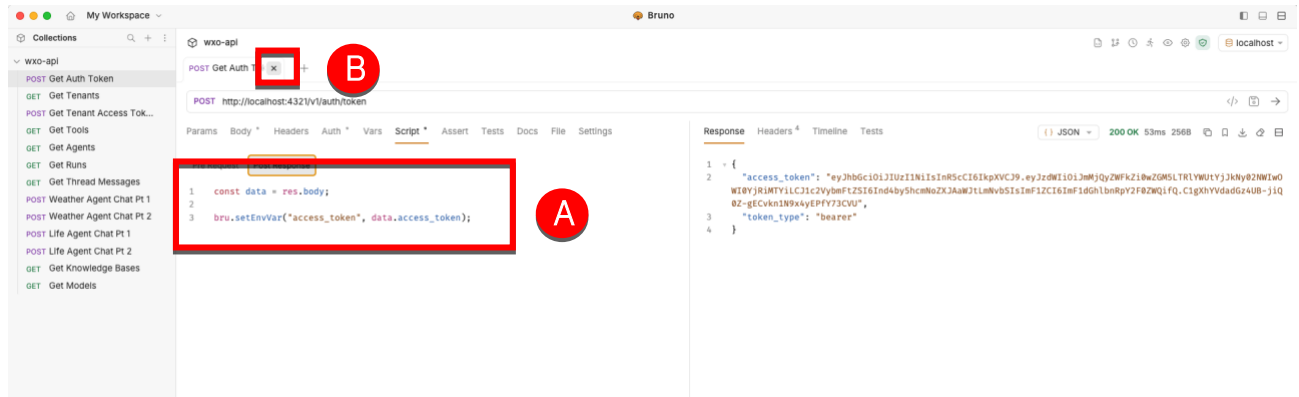
4. Once the response has completed, observe that the response body contains the access token (A). After that switch to the **Script** tab by clicking on it (B).



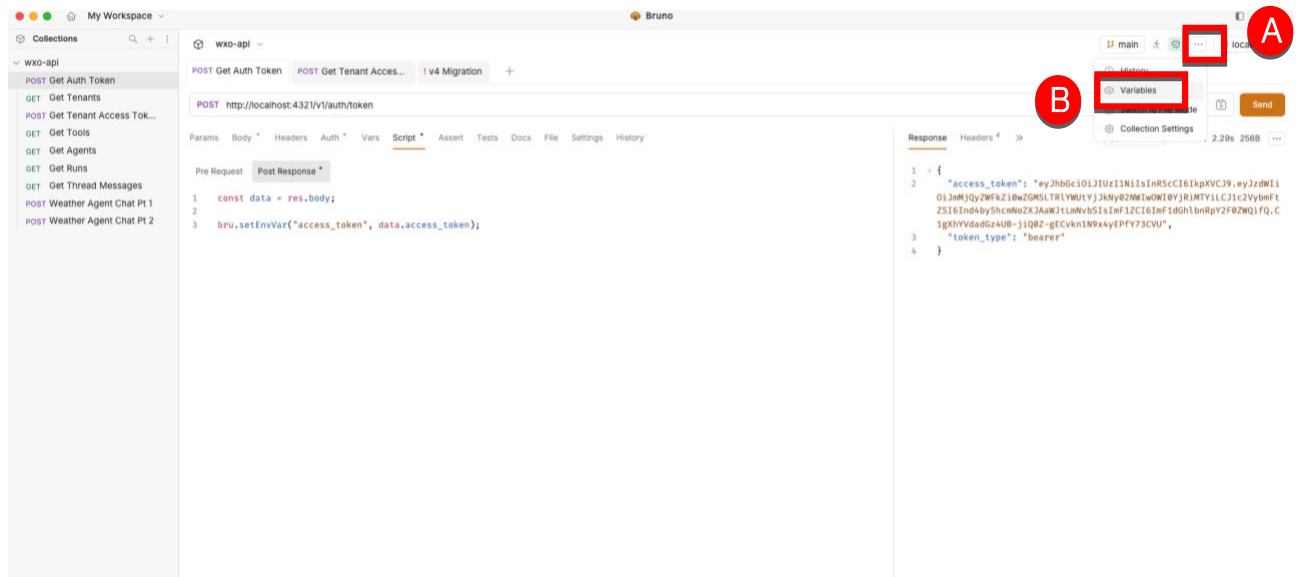
5. Click the **Post Response** tab (A).



6. Within the script tab observe that there is a **Post Response** script (A). This script takes the access token value from the response and saves it into an environment variable called **access_token** which is then used to include the token in later API requests. Close the **Get Auth Token** request panel (B).



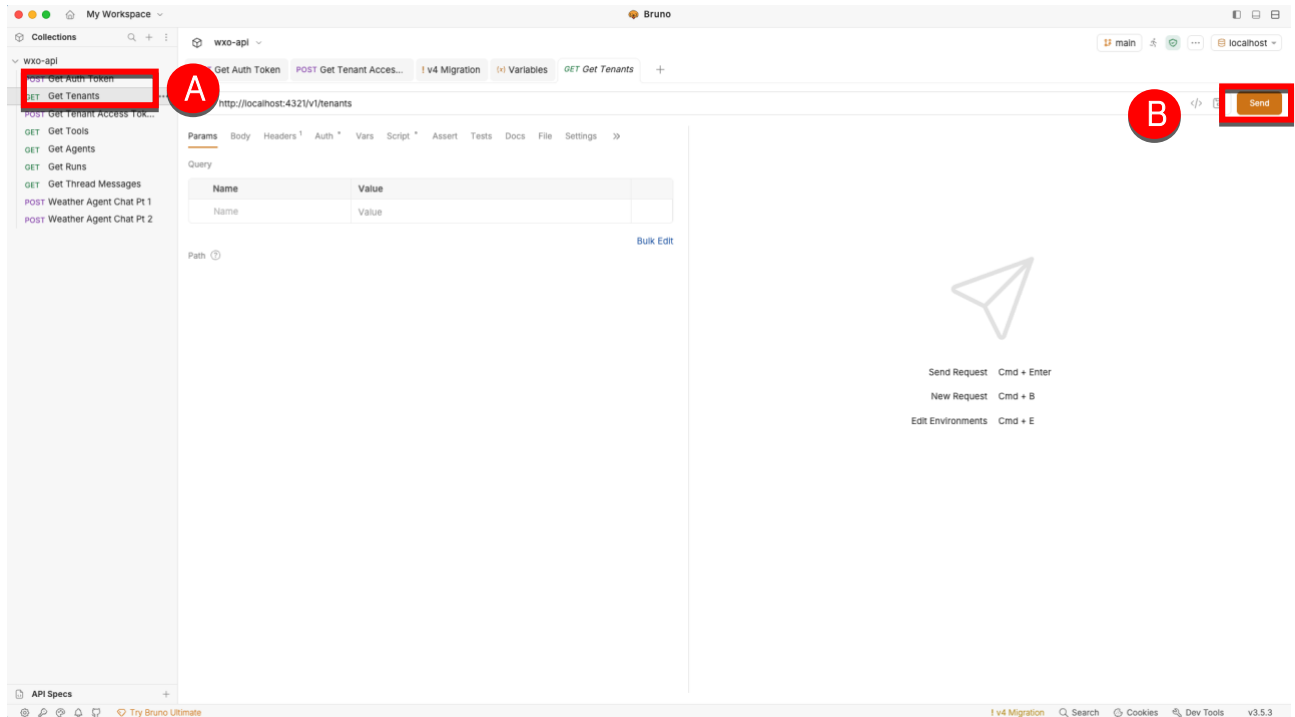
You can review any environment variables by clicking the ... icon (A) and then Variables (B).



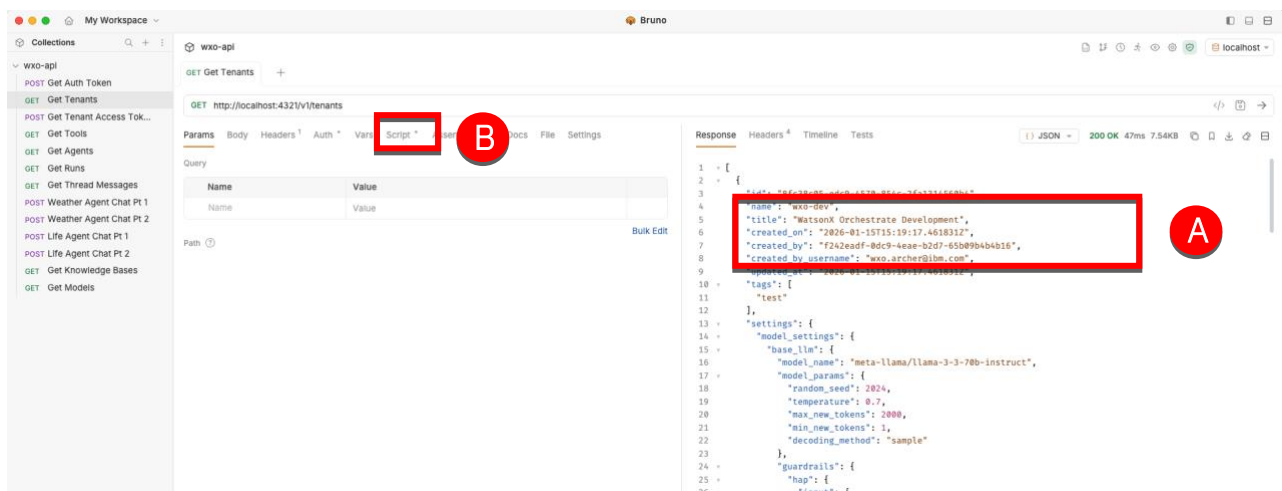
Listing tenants

In this section, we will be using Bruno to get the list of tenants available in your environment and saving the tenant details to be used in later requests.

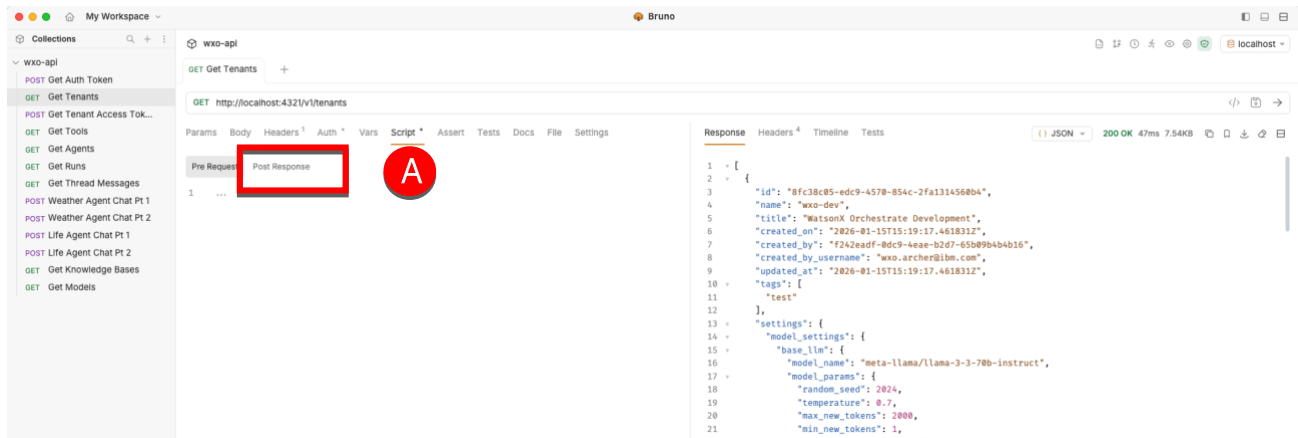
1. In the requests list, click the **Get Tenants** (A) item to open it, then click **Send** button (B).



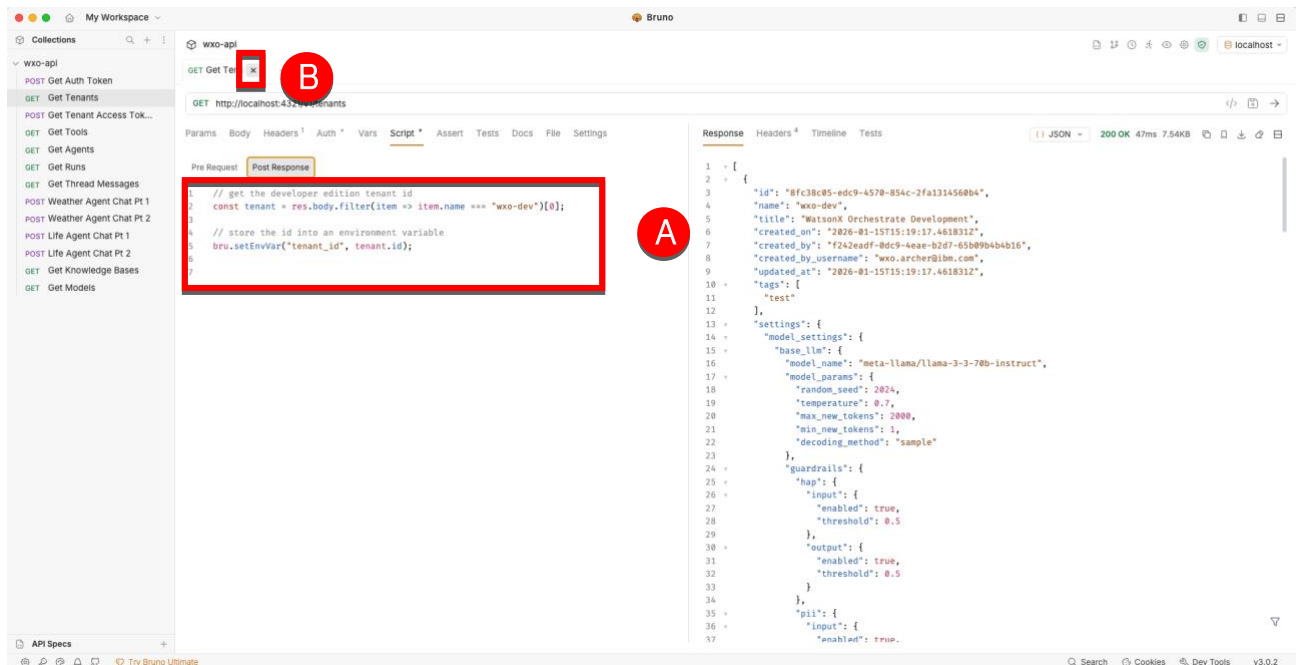
2. Once the response has completed, scroll down to locate the development tenant (identified with the name “wrxo-dev”) appears as an item in the array of results with its **id** (A). Click the **Script** tab (B) to open the script.



3. Click the **Post Response** tab (A).



4. Within the script tab observe that there is a **Post Response** script (A). This script takes the development tenant's id value from the response and saves it into an environment variable which is then used to include the tenant id in later API requests. Click the cross to close the **Get Tenants** request panel (B).

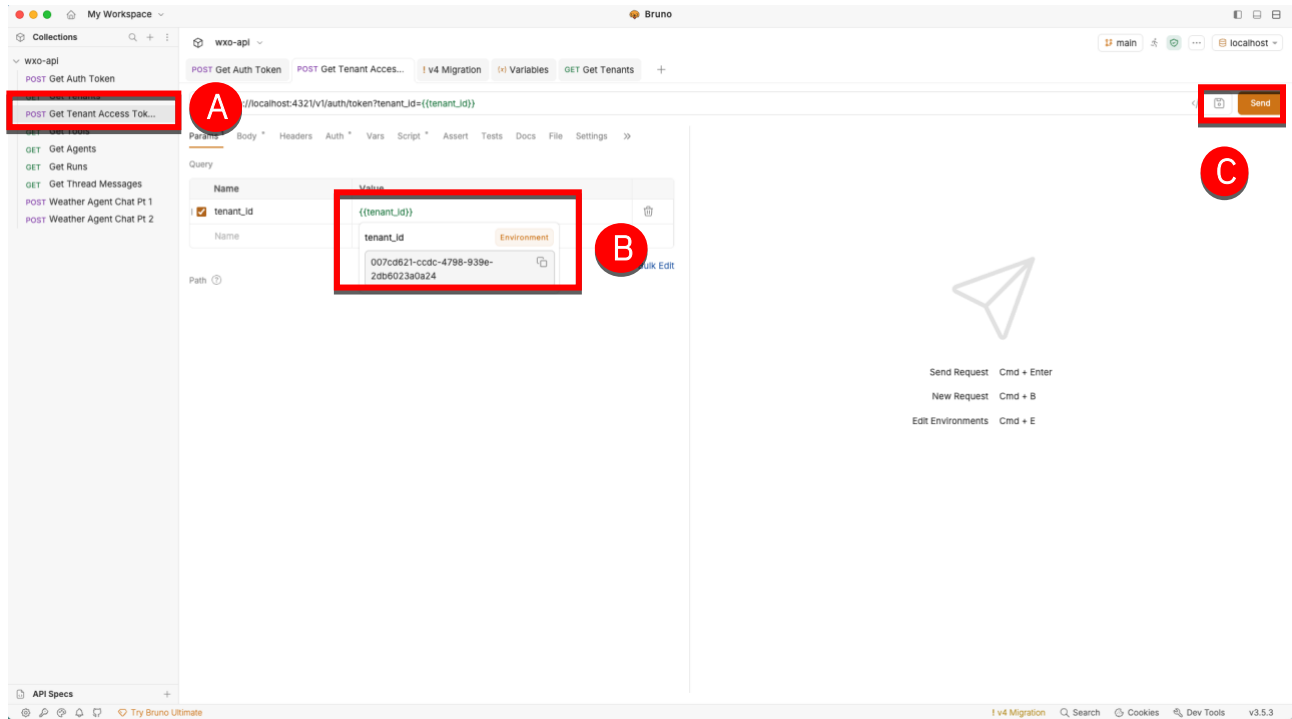


Note: The script finds the tenant id for the tenant called “wxo-dev”, any other tenants are ignored.

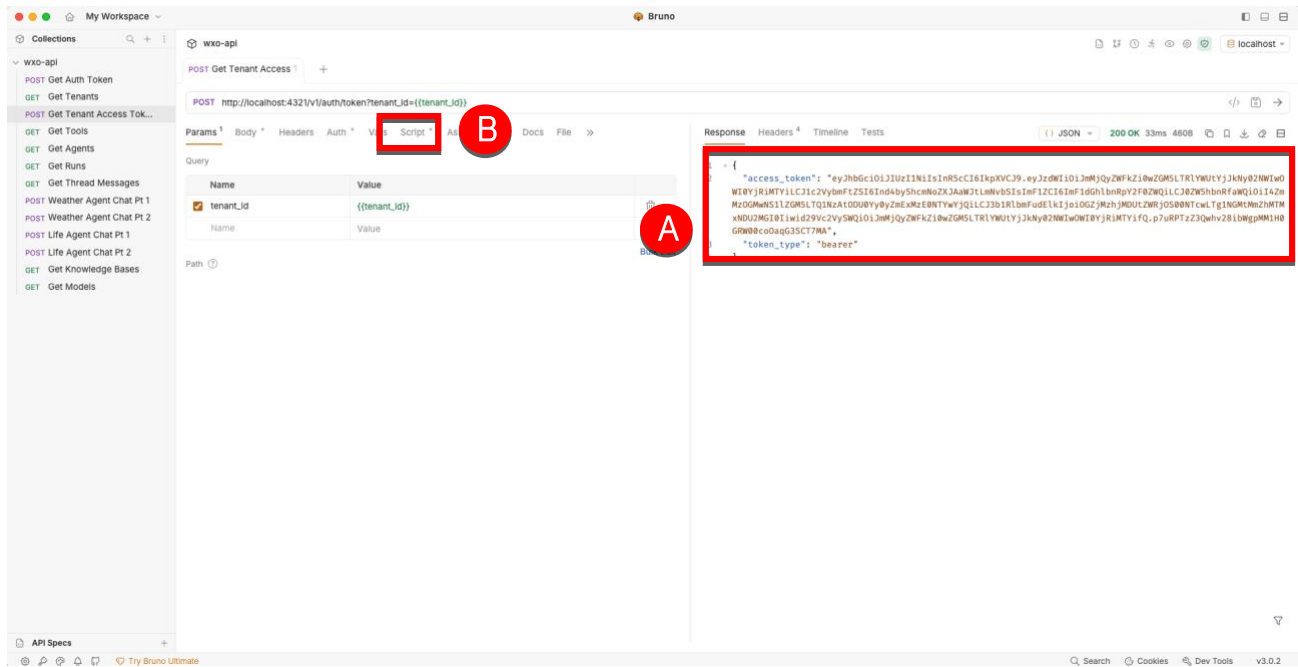
Getting a tenant-specific access token

In the former sections we have used Bruno to get the tenant `id`. Since some API requests require a tenant `id` to be included within the authorization token, in this section, we will be using Bruno to get & update the access token to include the default tenant.

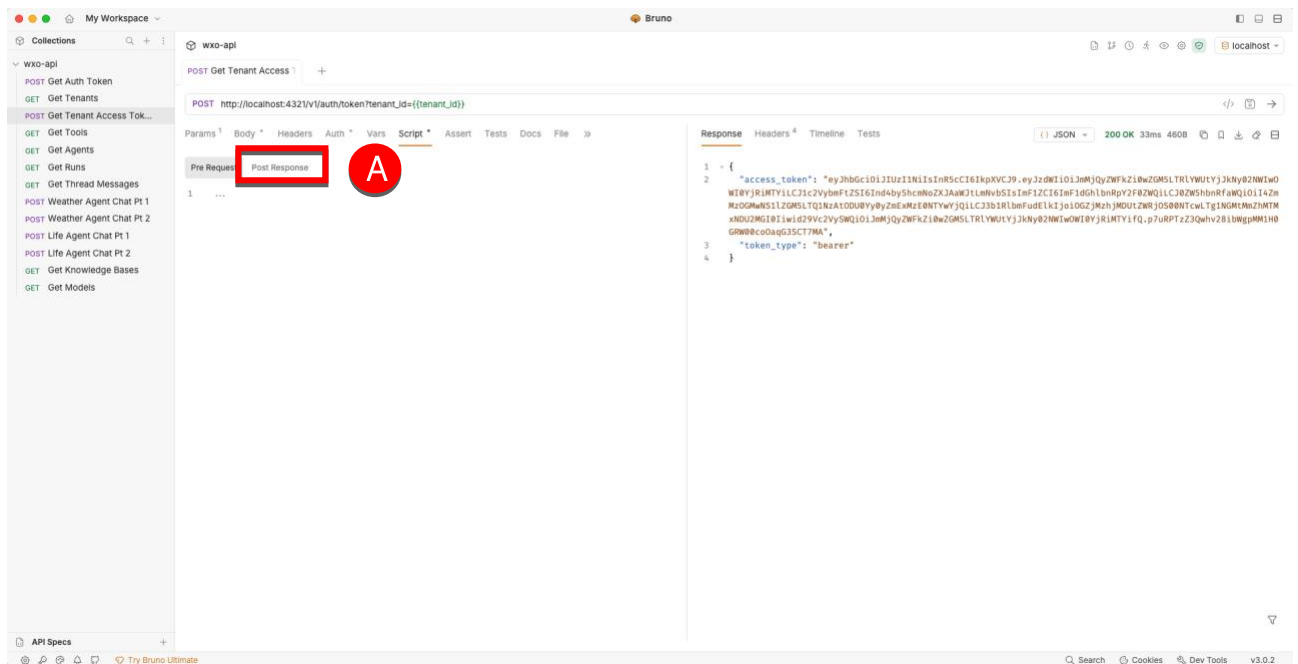
1. In the requests list, click the **Get Tenant Access Token** (A) item to open it. Within the Params tab we can see that the `tenant_id` parameter value is being set by an environment variable (B) (double curly braces are used to access variables in Bruno). Click **Send** button (C).



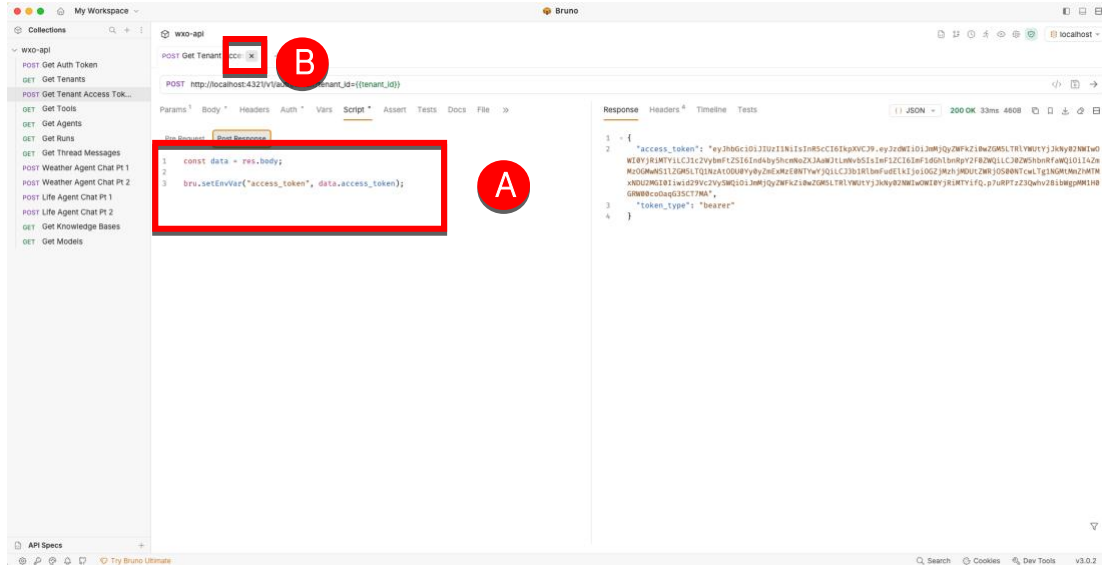
2. Once the response has completed, observe that the response body contains the **access_token** (A). Click the **Script** tab (B) to open the script.



3. Click the **Post Response** tab (A).



4. Within the script tab observe that there is a **Post Response** script (A). Just like in **Get Auth Token's** Post Response script, this script takes the access token value from the response and updates the same environment variable which is then used to include the token in later API requests. Click the cross (B) to close the **Get Tenant Access Token** request panel.

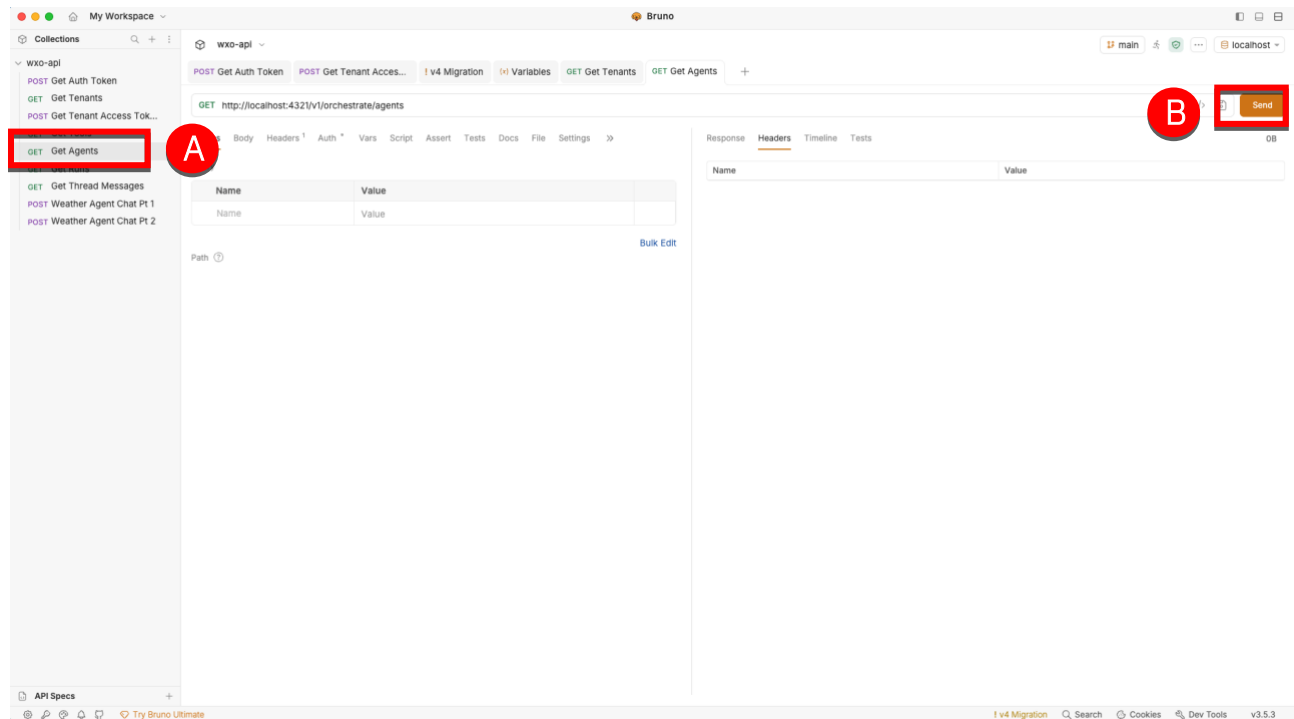


Now all further requests will be authorized using the tenant-specific access token that is saved in an environment variable within Bruno.

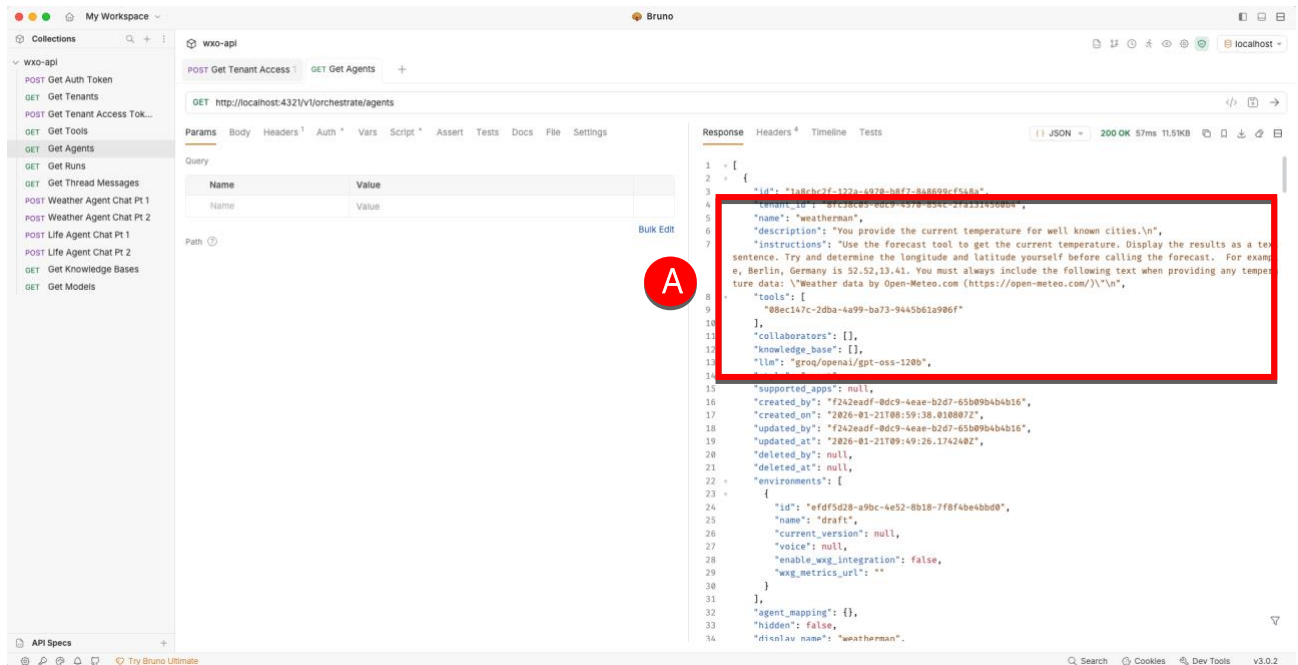
Listing registered agents

In this section, we will be querying the API to get the list of agents available in your environment. Agents registered in your environment can be used in the chat interface and can be referenced by other agents for agentic collaboration within same tenant.

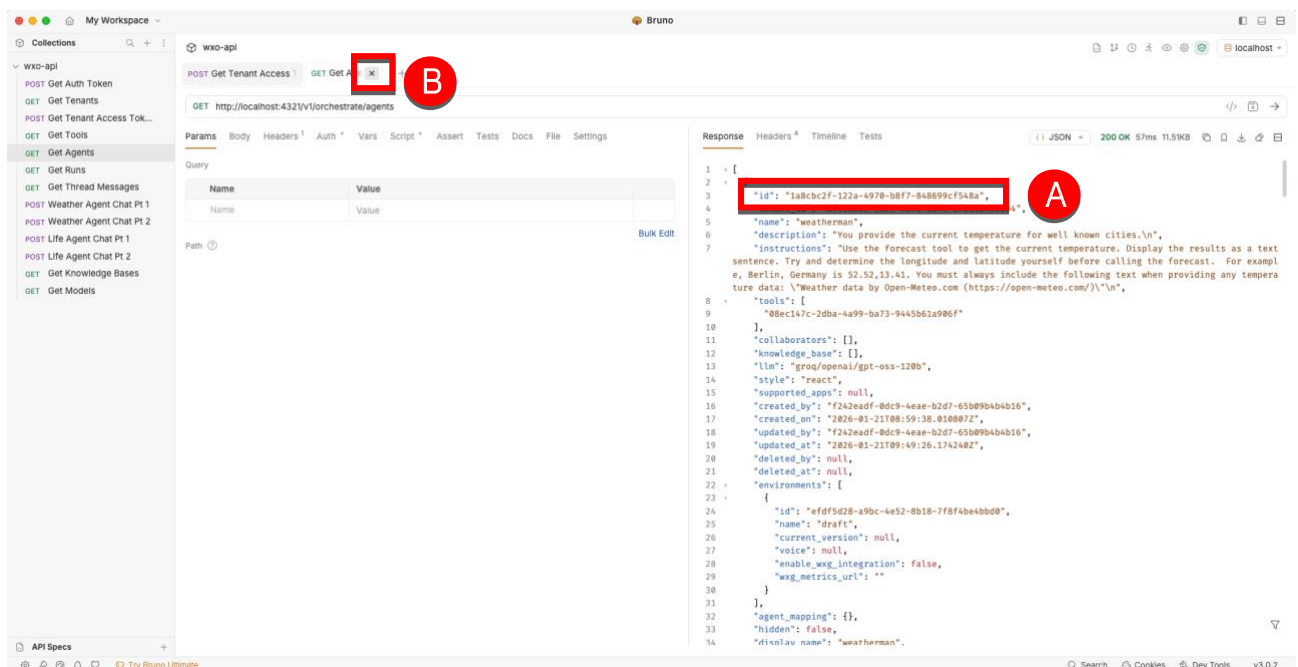
1. Locate the **Get Agents** (A) and click it to open its details. Next click the **Send** button (B) which will send an HTTP GET request to the API and wait for the response to complete.



2. In the response, structured as an array, we can observe an item for the agent identified by the name **weatherman** (A), which we created earlier.



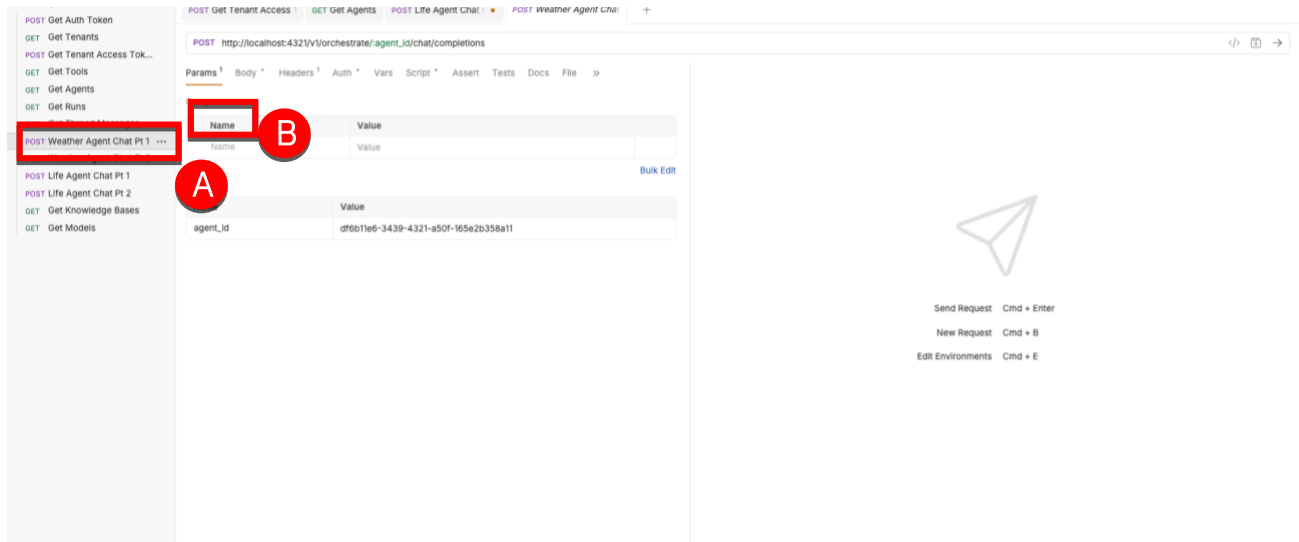
3. Next, copy the agent **id** (A) for the agent named **weatherman** from the response body into your notes. We will use that information in a later section. Click the cross (B) to close the **Get Agents** panel (B).



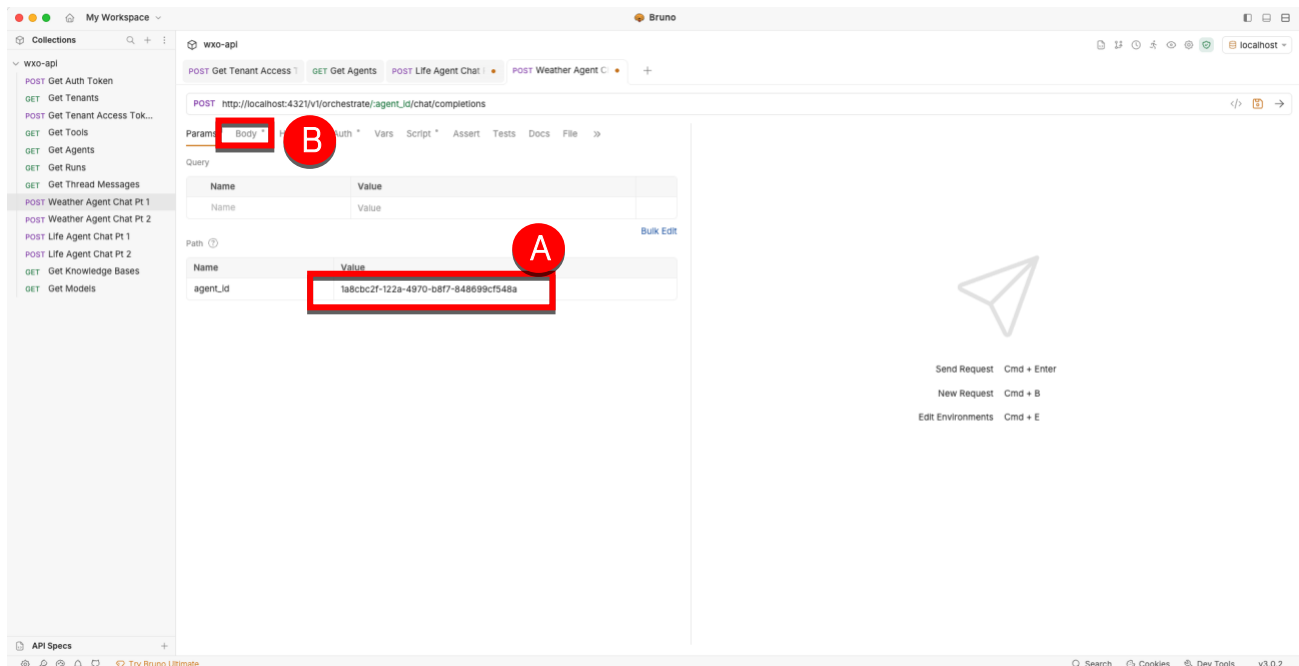
Chatting with an agent

In this section we will use the agent information obtained in previous step to engage in a chat with the agent by using API request.

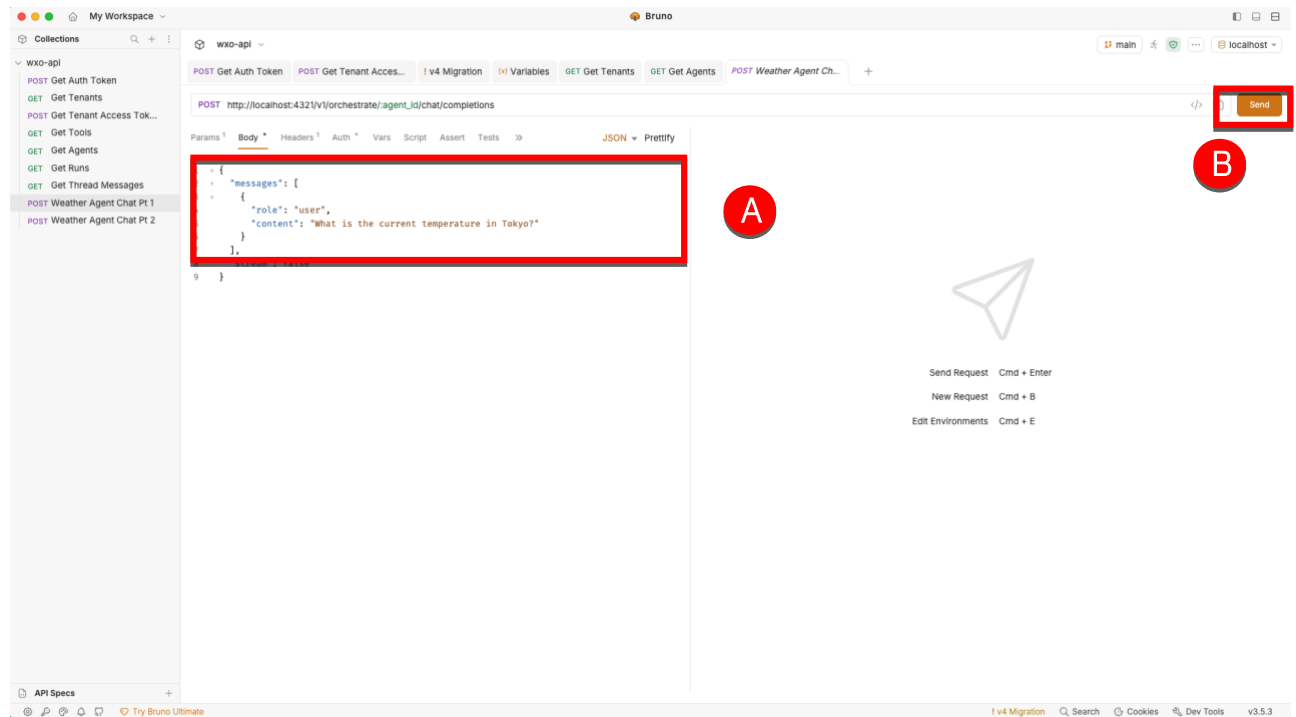
1. In the requests list, click the **Weather Agent Chat Pt 1** (A) item to open it and select the **Params** tab (B).



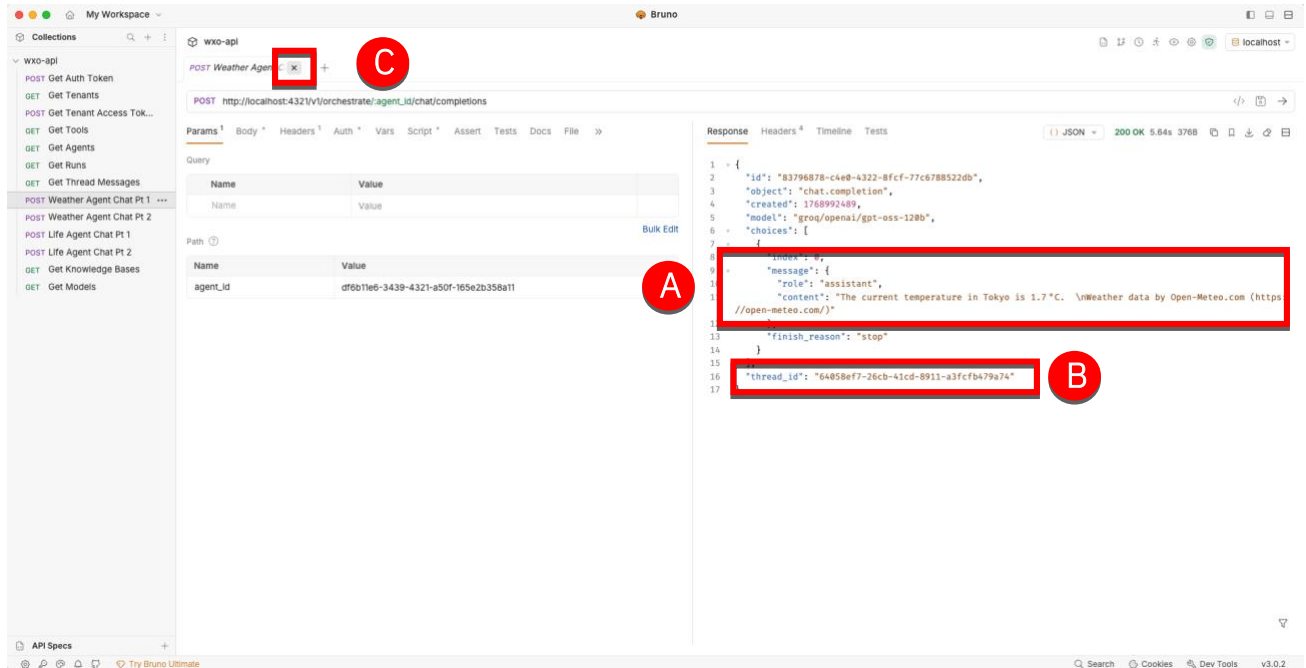
2. A path parameter called **agent_id** is defined. In the **value** field (A) enter the agent id you copied in [Listing Registered Agents](#), then switch to the **Body** tab by clicking it (B).



3. Within the body tab observe the format of the request which includes an array of **messages**. Each message object (A) contains a **role** field – in this case a **"user"** – and a **content** field which includes the message contents. After that click the **Send** (B) which will send an HTTP POST request.

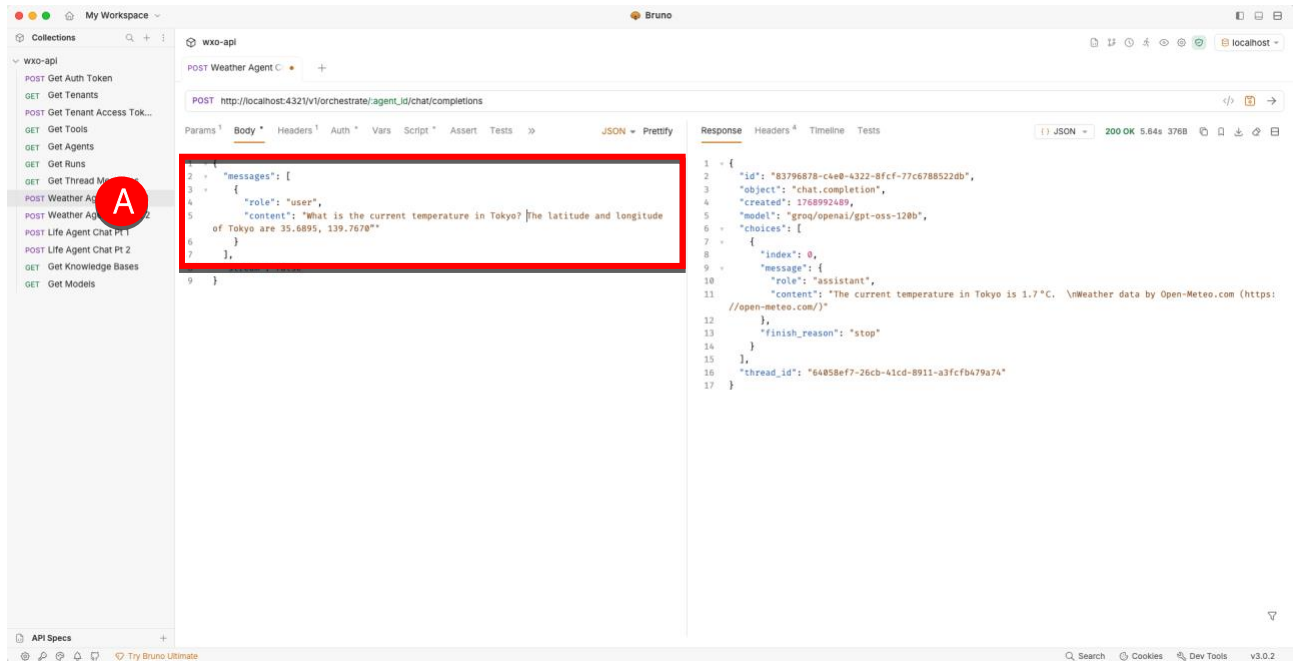


4. Once the request has completed, locate the **message** object containing the response of the agent (A). Next, observe that the response also includes a **thread_id** value, which is used to identify the conversation thread (B), copy the **thread_id** value into your notes as we will use it in a later section. After that, close the **Weather Agent Chat Pt 1** panel (C).



5. If the agent doesn't return a temperature on the first attempt, modify message content to include longitude and latitude (A).

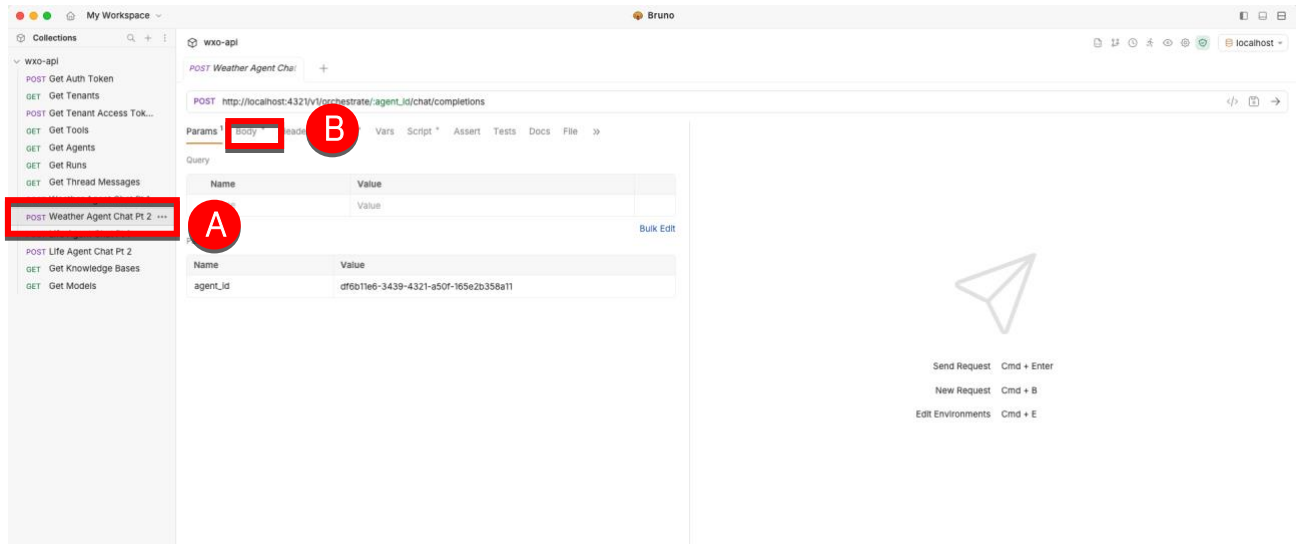
"What is the current temperature in Tokyo? The latitude and longitude of Tokyo are 35.6895, 139.7670"



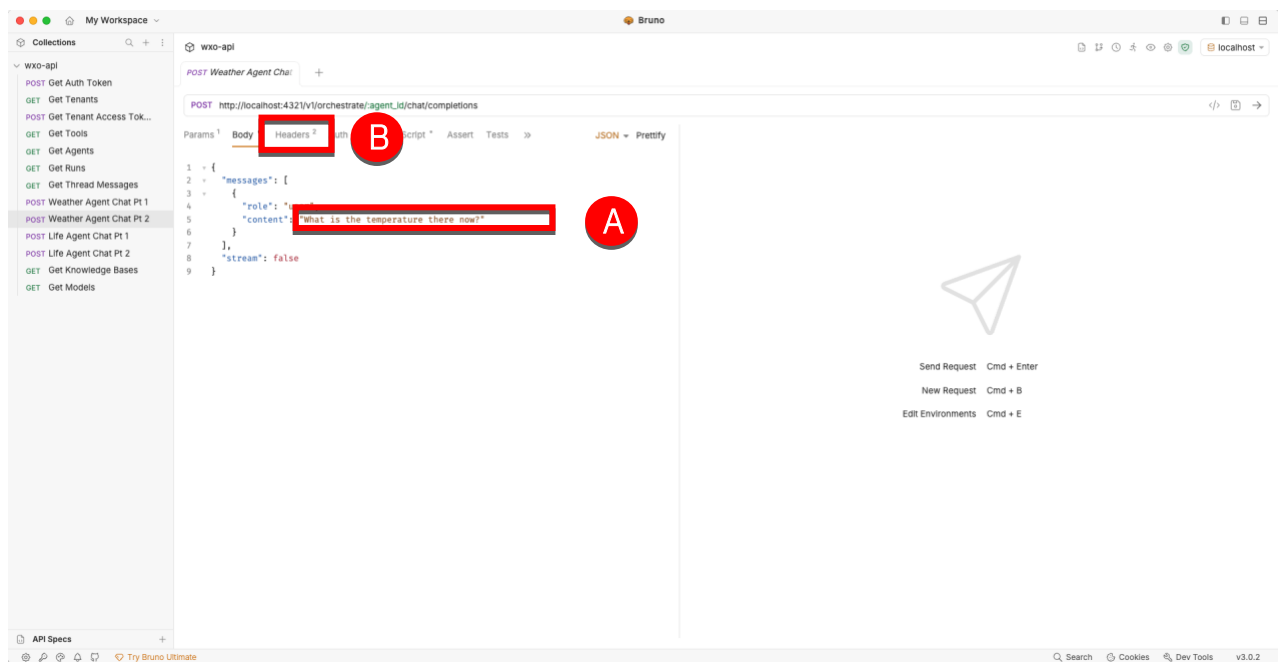
Continuing a chat with an agent

In this section we will use the thread information obtained in previous step to continue the chat conversation with the agent by modifying the API request to refer to specific conversation thread.

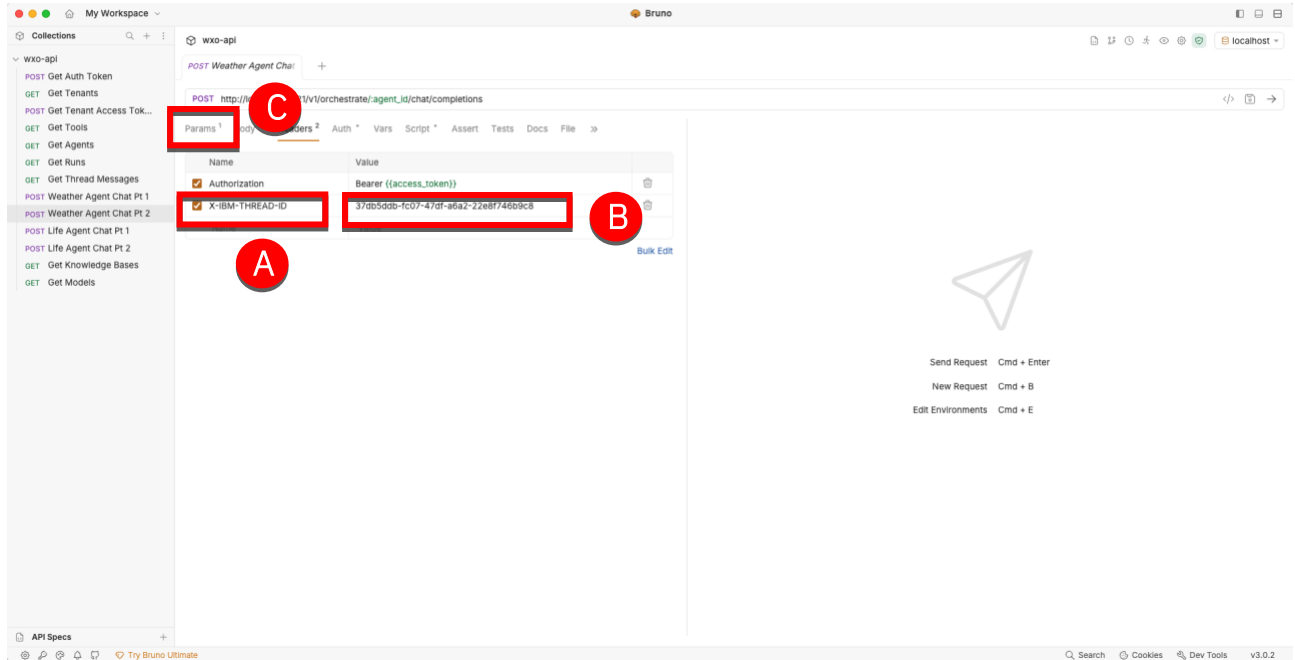
1. In the requests list, click the **Weather Agent Chat Pt 2** (A) item to open it then within it, switch to the **Body** tab by clicking it (B).



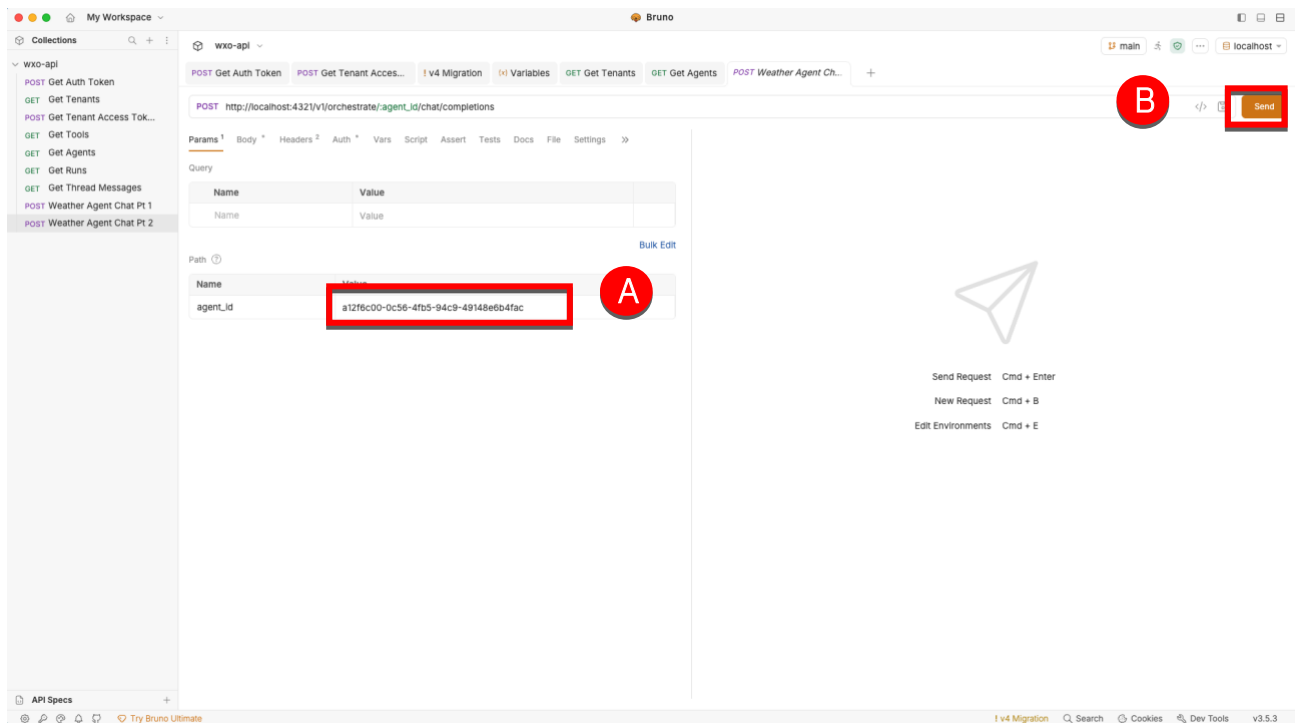
2. Within the Body tab, observe the **content** field value with the text “**what is the temperature there now?**” (A). Note that the message content does not specify neither a city name nor a location. After that, switch to the **Headers** tab by clicking it (B).



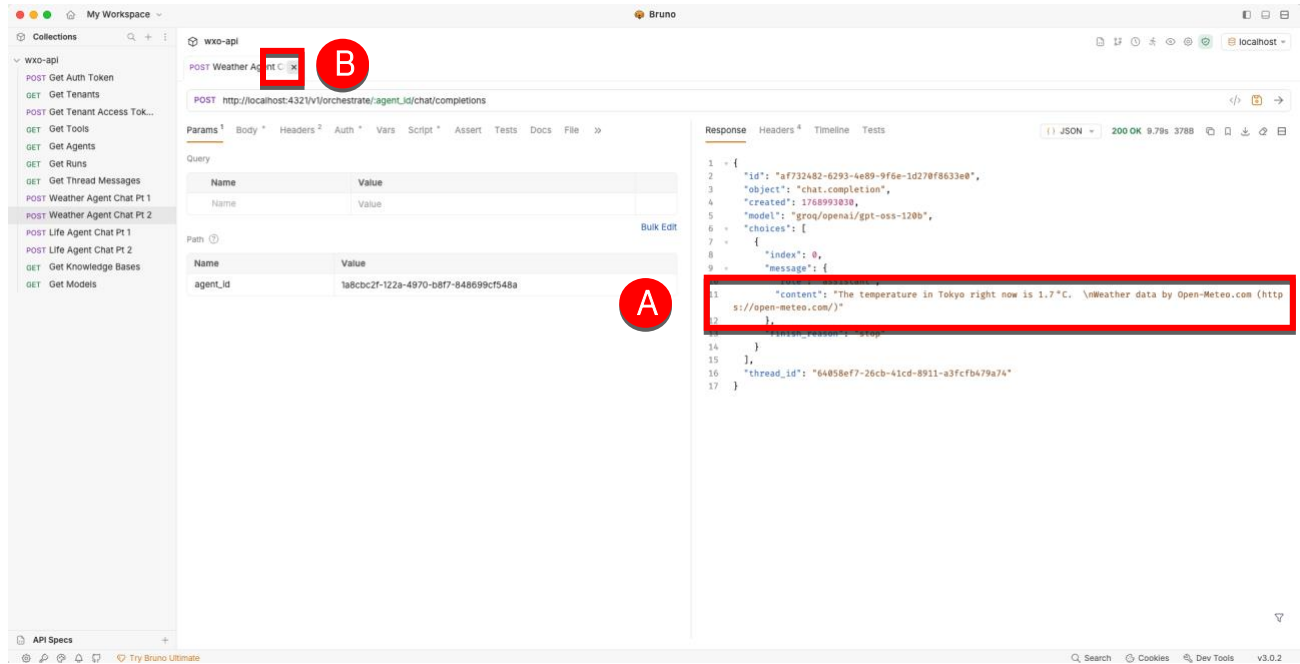
- Within the Headers tab, locate the Key column field with the Key "X-IBM-THREAD-ID" (A), then set the Value column field to the value of the thread id you have saved in the former step (B). After that, click the Params tab (C).



- Copy the agent id you have saved in your notes into the **agent_id** value cell (A) then click the **Send** button (B).



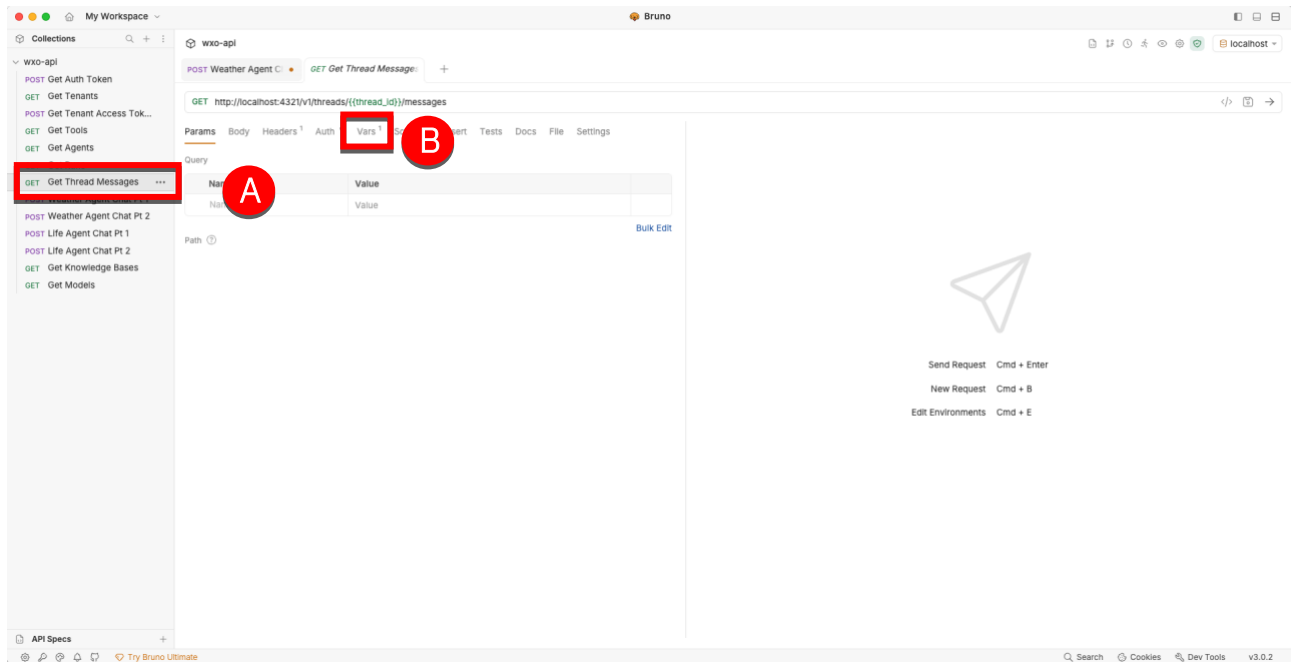
5. In the response we can see that the agent replied properly relying on the location information from the conversation thread (A). Using the thread id, the former conversation messages were included as part of the context given to the agent together with the user's new query. This provided the agent with the location we are referring to and without requiring the agent to ask for it. Click the cross (B) to close the **Weather Agent Chat Pt 2** panel.



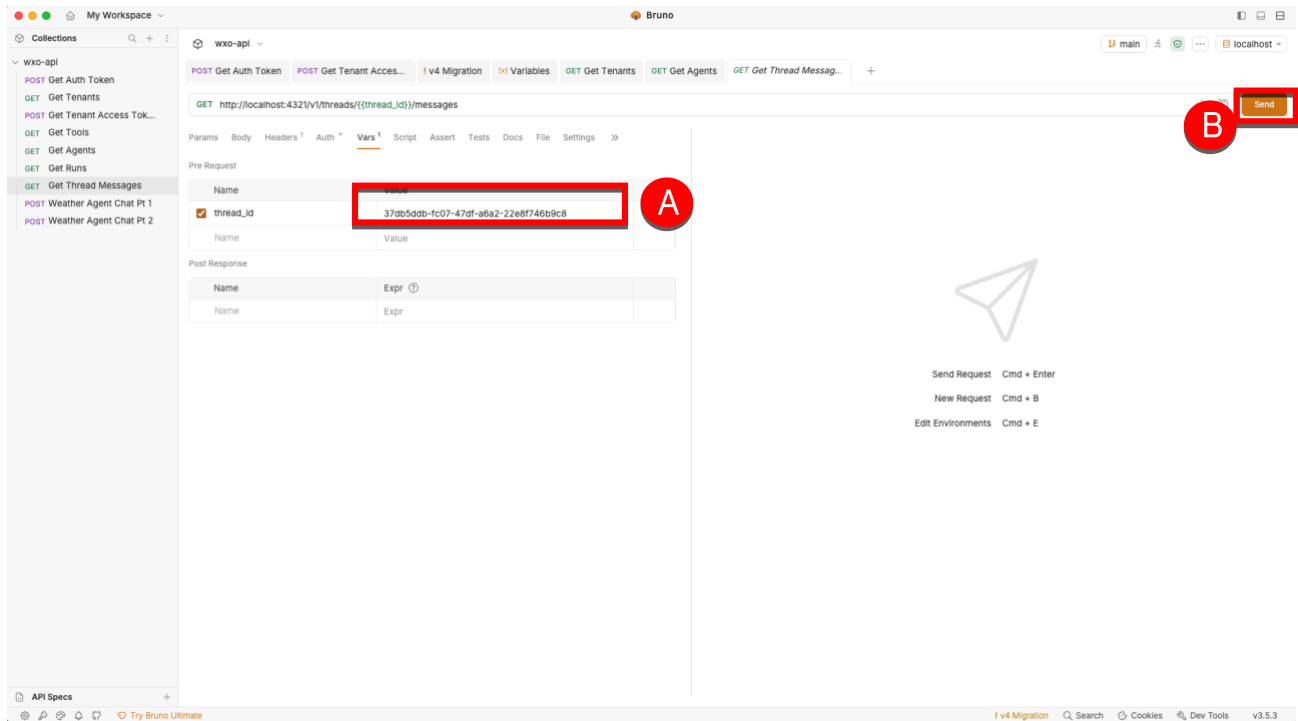
Fetch a thread of messages

Whenever a chat conversation takes place in your environment watsonx Orchestrate stores the conversation details and message threads. In this section, we will be querying the API to get the list of chat conversations that have taken place in your environment.

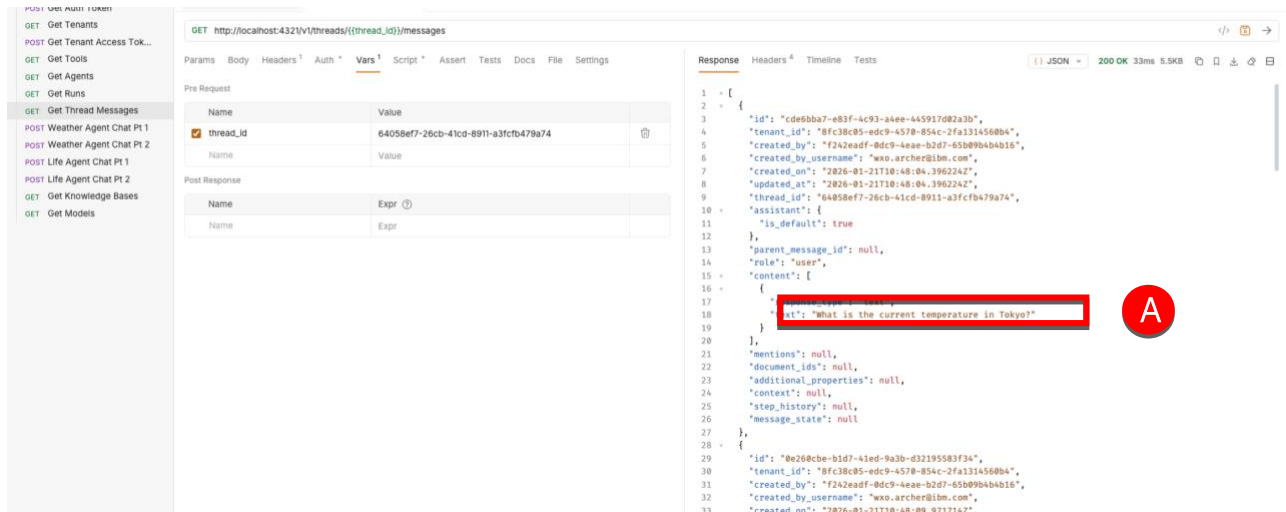
1. In the requests list, click the **Get Thread Messages** (A) item to open and then within it open the **Vars** tab by clicking it (B).



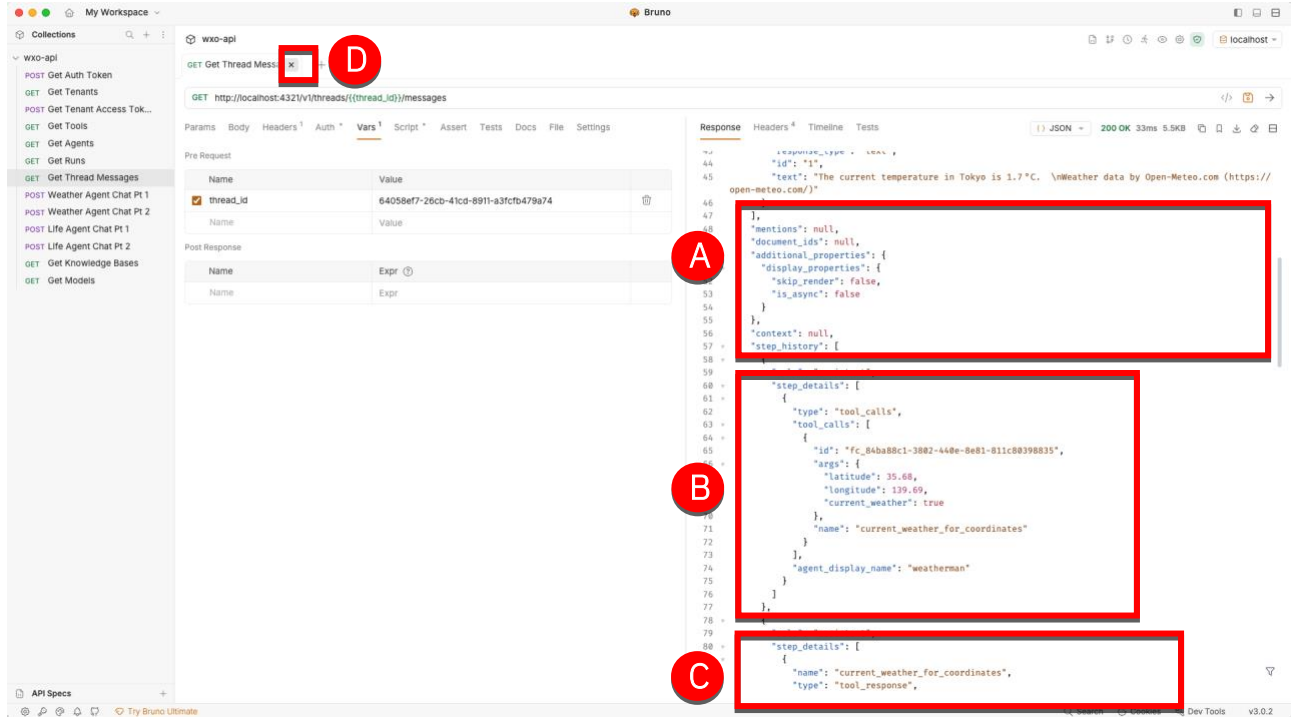
- This request includes a variable value named **thread_id**. In the **value** field (A) enter the thread id you copied in [Chatting With An Agent](#), then click execute (B).



- Once the response completes, within the response panel we can see all the messages exchanged in the conversation, starting with the user's initial message (A).



4. Within the agent response locate the **step_history** element (A) which contains the steps the agent took to process the response, here we can observe steps taken by the agent to answer the user's query, including the usage of the **forecast** tool by the agent, the tool call (B) and the tool response (C). Lastly, close the **Get Thread Messages** panel (D).



Summary

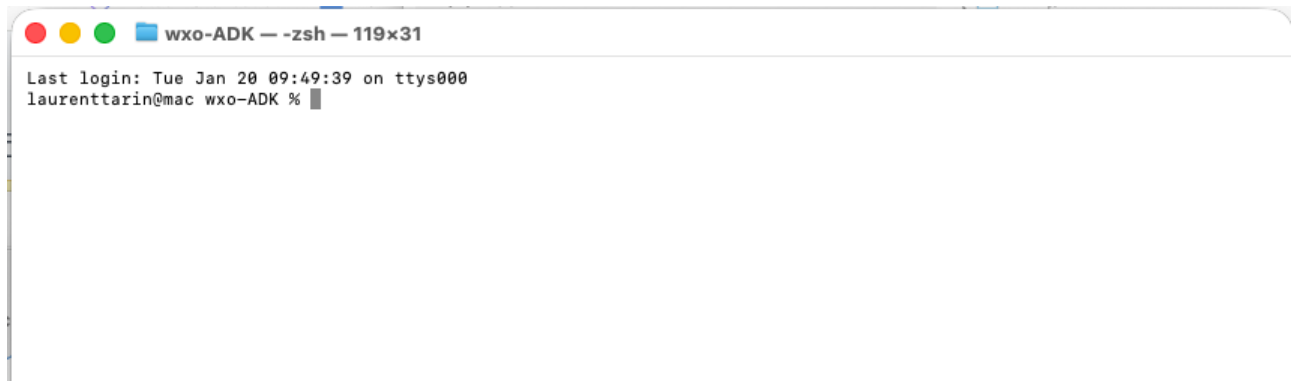
In this lab you have learned how to connect and use the APIs offered by watsonx Orchestrate using the built-in API documentation. This documentation is provided when using a local Developer Edition server, but it can become cumbersome, so we also used an API testing tool.

In real-world environments developers can use watsonx Orchestrate to develop agents and integrate them with their own solutions using the APIs used in this lab.

Appendix: Reset your environment

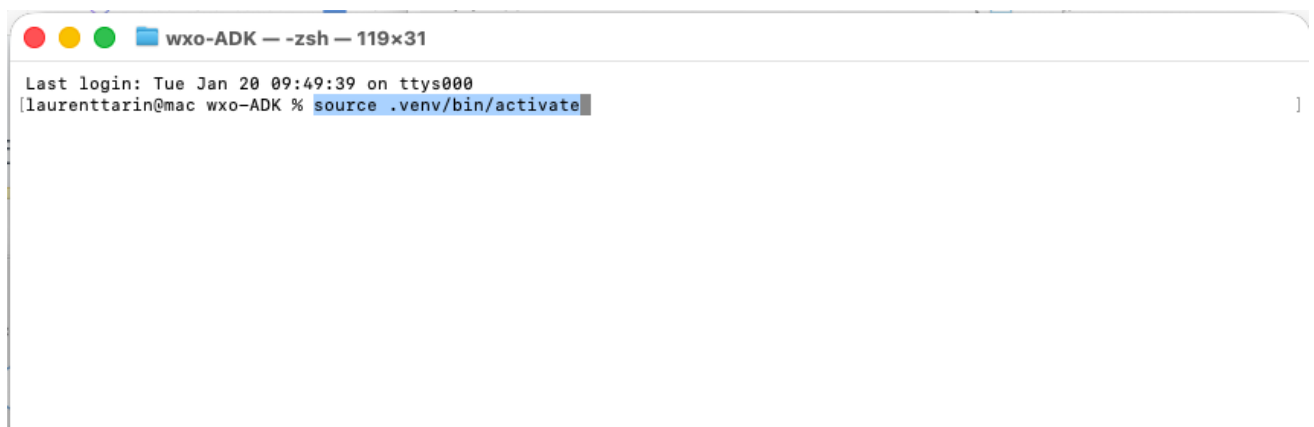
These steps will remove any existing tools and agents from your environment and restart the local watsonx Orchestrate server in readiness to start the lab.

1. Open a Terminal window on your **wxO-ADK** folder (A).



2. Enter the following command into the terminal to start your virtual environment:

```
source .venv/bin/activate
```



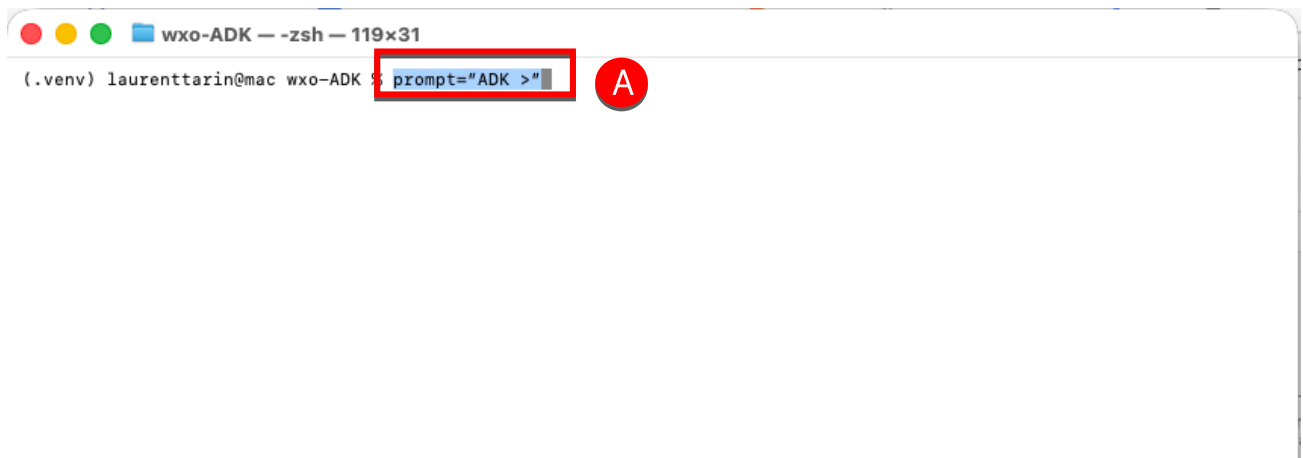
3. Check that the prompt starts with (.venv) (A) indicating that you are in your virtual environment.



A terminal window titled "wxo-ADK --zsh-- 119x31". The output shows the last login time and the execution of the command `source .venv/bin/activate`. The prompt has changed from `laurenttarin@mac wxo-ADK %` to `(.venv) laurenttarin@mac wxo-ADK %`, indicating the virtual environment is active.

4. Enter the following command into the terminal to improve the command visibility, then press Enter:

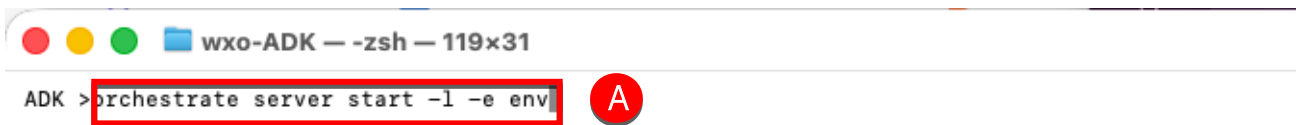
```
MS Windows users: function prompt{"ADK>"}  
Mac users: prompt="ADK >"
```



A terminal window titled "wxo-ADK --zsh-- 119x31". The prompt is `(.venv) laurenttarin@mac wxo-ADK %`. The command `prompt="ADK >"` is being entered and is highlighted with a red box. A red circle with the letter 'A' is next to the command, indicating it is the correct command to enter.

5. Enter the following command into the terminal to start the server with your env configuration, then press Enter.

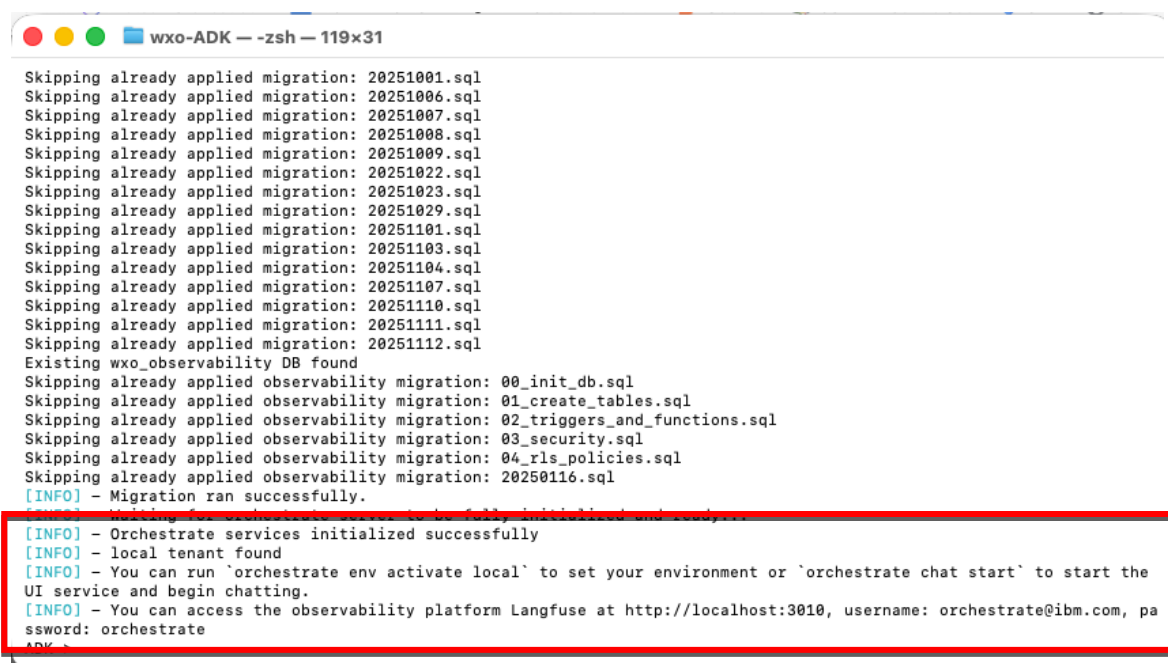
```
orchestrate server start -l -e env
```



```
wxo-ADK — -zsh — 119x31
ADK > orchestrate server start -l -e env
```

Note: Some services may fail to start on the first attempt, and you may encounter issues connecting to the docker repository. Retry the start command in this situation, depending on the volume of requests, it can take several attempts to connect.

When startup is complete the terminal output will resemble the example below, restart can take 2-3 minutes to complete. The local server will be ready when you will see this **INFO** in the terminal screen (A):



```
wxo-ADK — -zsh — 119x31
Skipping already applied migration: 20251001.sql
Skipping already applied migration: 20251006.sql
Skipping already applied migration: 20251007.sql
Skipping already applied migration: 20251008.sql
Skipping already applied migration: 20251009.sql
Skipping already applied migration: 20251022.sql
Skipping already applied migration: 20251023.sql
Skipping already applied migration: 20251029.sql
Skipping already applied migration: 20251101.sql
Skipping already applied migration: 20251103.sql
Skipping already applied migration: 20251104.sql
Skipping already applied migration: 20251107.sql
Skipping already applied migration: 20251110.sql
Skipping already applied migration: 20251111.sql
Skipping already applied migration: 20251112.sql
Existing wxo_observability DB found
Skipping already applied observability migration: 00_init_db.sql
Skipping already applied observability migration: 01_create_tables.sql
Skipping already applied observability migration: 02_triggers_and_functions.sql
Skipping already applied observability migration: 03_security.sql
Skipping already applied observability migration: 04_rls_policies.sql
Skipping already applied observability migration: 20250116.sql
[INFO] - Migration ran successfully.
[INFO] - Starting local observability platform fully initialized and ready...
[INFO] - Orchestrate services initialized successfully
[INFO] - local tenant found
[INFO] - You can run 'orchestrate env activate local' to set your environment or 'orchestrate chat start' to start the UI service and begin chatting.
[INFO] - You can access the observability platform Langfuse at http://localhost:3010, username: orchestrate@ibm.com, password: orchestrate
```

You are now ready to work with watsonx Orchestrate ADK.

